Namespace FractPal.API.Controllers

Classes

AuthController

Handles authentication operations including user login and registration.

CommentsController

Handles comment CRUD operations on posts. All endpoints require an authenticated user.

FractalController

Handles all fractal CRUD operations, feed retrieval, forking, and social interactions (likes). All endpoints require an authenticated user.

PostsController

Handles post lifecycle operations including feed retrieval, publishing and unpublishing fractals as posts, and social interactions (likes). All endpoints require an authenticated user.

UserController

Handles user profile and social operations such as viewing profiles, updating bio, following/unfollowing users, and searching for users.

Class AuthController

Namespace: FractPal.API.Controllers

Assembly: FractPal.API.dll

Handles authentication operations including user login and registration.

```
[Route("api/[controller]")]
[ApiController]
public class AuthController : ControllerBase
```

Inheritance

object ← ControllerBase ← AuthController

Inherited Members

ControllerBase.StatusCode(int) , ControllerBase.StatusCode(int, object) ,
ControllerBase.Content(string) , ControllerBase.Content(string, string) ,
ControllerBase.Content(string, string, Encoding) ,
ControllerBase.Content(string, MediaTypeHeaderValue) , ControllerBase.NoContent() ,
ControllerBase.Ok() , ControllerBase.Ok(object) , ControllerBase.Redirect(string) ,
ControllerBase.RedirectPermanent(string) , ControllerBase.RedirectPreserveMethod(string) ,
ControllerBase.RedirectPermanentPreserveMethod(string) ,
ControllerBase.LocalRedirect(string) , ControllerBase.LocalRedirectPermanent(string) ,
ControllerBase.LocalRedirectPreserveMethod(string) ,
ControllerBase.LocalRedirectPermanentPreserveMethod(string) ,
ControllerBase.RedirectToAction() , ControllerBase.RedirectToAction(string) ,
ControllerBase.RedirectToAction(string, object) ,
ControllerBase.RedirectToAction(string, string) ,
ControllerBase.RedirectToAction(string, string, object) ,
ControllerBase.RedirectToAction(string, string, string) ,
ControllerBase.RedirectToAction(string, string, object, string) ,
ControllerBase.RedirectToActionPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToActionPermanent(string) ,
ControllerBase.RedirectToActionPermanent(string, object) ,
ControllerBase.RedirectToActionPermanent(string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, object) ,
ControllerBase.RedirectToActionPermanent(string, string, object, string) ,
ControllerBase.RedirectToActionPermanentPreserveMethod(string, string, object, string) ,

ControllerBase.RedirectToRoute(string) , ControllerBase.RedirectToRoute(object) ,
ControllerBase.RedirectToRoute(string, object) ,
ControllerBase.RedirectToRoute(string, string) ,
ControllerBase.RedirectToRoute(string, object, string) ,
ControllerBase.RedirectToRoutePreserveMethod(string, object, string) ,
ControllerBase.RedirectToRoutePermanent(string) ,
ControllerBase.RedirectToRoutePermanent(object) ,
ControllerBase.RedirectToRoutePermanent(string, object) ,
ControllerBase.RedirectToRoutePermanent(string, string) ,
ControllerBase.RedirectToRoutePermanent(string, object, string) ,
ControllerBase.RedirectToRoutePermanentPreserveMethod(string, object, string) ,
ControllerBase.RedirectToPage(string) , ControllerBase.RedirectToPage(string, object) ,
ControllerBase.RedirectToPage(string, string) ,
ControllerBase.RedirectToPage(string, string, object) ,
ControllerBase.RedirectToPage(string, string, string) ,
ControllerBase.RedirectToPage(string, string, object, string) ,
ControllerBase.RedirectToPagePermanent(string) ,
ControllerBase.RedirectToPagePermanent(string, object) ,
ControllerBase.RedirectToPagePermanent(string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, object, string) ,
ControllerBase.RedirectToPagePreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToPagePermanentPreserveMethod(string, string, object, string) ,
ControllerBase.File(byte[], string) , ControllerBase.File(byte[], string, bool) ,
ControllerBase.File(byte[], string, string) , ControllerBase.File(byte[], string, string, bool) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string) , ControllerBase.File(Stream, string, bool) ,
ControllerBase.File(Stream, string, string) , ControllerBase.File(Stream, string, string, bool) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string) , ControllerBase.File(string, string, bool) ,
ControllerBase.File(string, string, string) , ControllerBase.File(string, string, string, bool) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,

ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string) , ControllerBase.PhysicalFile(string, string, bool) ,
ControllerBase.PhysicalFile(string, string, string) ,
ControllerBase.PhysicalFile(string, string, string, bool) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.Unauthorized() , ControllerBase.Unauthorized(object) ,
ControllerBase.NotFound() , ControllerBase.NotFound(object) , ControllerBase.BadRequest() ,
ControllerBase.BadRequest(object) , ControllerBase.BadRequest(ModelStateDictionary) ,
ControllerBase.UnprocessableEntity() , ControllerBase.UnprocessableEntity(object) ,
ControllerBase.UnprocessableEntity(ModelStateDictionary) , ControllerBase.Conflict() ,
ControllerBase.Conflict(object) , ControllerBase.Conflict(ModelStateDictionary) ,
ControllerBase.Problem(string, string, int?, string, string) ,
ControllerBase.Problem(string, string, int?, string, string, IDictionary<string, object>) ,
ControllerBase.ValidationProblem(ValidationProblemDetails) ,
ControllerBase.ValidationProblem(ModelStateDictionary) , ControllerBase.ValidationProblem() ,
ControllerBase.ValidationProblem(string, string, int?, string, string, ModelStateDictionary) ,
ControllerBase.ValidationProblem(string, string, int?, string, string, ModelStateDictionary,
IDictionary<string, object>) ,
ControllerBase.Created() , ControllerBase.Created(string, object) ,
ControllerBase.Created(Uri, object) , ControllerBase.CreatedAtAction(string, object) ,
ControllerBase.CreatedAtAction(string, object, object) ,
ControllerBase.CreatedAtAction(string, string, object, object) ,
ControllerBase.CreatedAtRoute(string, object) , ControllerBase.CreatedAtRoute(object, object) ,
ControllerBase.CreatedAtRoute(string, object, object) , ControllerBase.Accepted() ,
ControllerBase.Accepted(object) , ControllerBase.Accepted(Uri) ,
ControllerBase.Accepted(string) , ControllerBase.Accepted(string, object) ,
ControllerBase.Accepted(Uri, object) , ControllerBase.AcceptedAtAction(string) ,
ControllerBase.AcceptedAtAction(string, string) ,
ControllerBase.AcceptedAtAction(string, object) ,
ControllerBase.AcceptedAtAction(string, string, object) ,
ControllerBase.AcceptedAtAction(string, object, object) ,
ControllerBase.AcceptedAtAction(string, string, object, object) ,
ControllerBase.AcceptedAtRoute(object) , ControllerBase.AcceptedAtRoute(string) ,
ControllerBase.AcceptedAtRoute(string, object) ,
ControllerBase.AcceptedAtRoute(object, object) ,
ControllerBase.AcceptedAtRoute(string, object, object) , ControllerBase.Challenge() ,
ControllerBase.Challenge(params string[]) ,

ControllerBase.Challenge(AuthenticationProperties) ,
ControllerBase.Challenge(AuthenticationProperties, params string[]) , ControllerBase.Forbid() ,
ControllerBase.Forbid(params string[]) , ControllerBase.Forbid(AuthenticationProperties) ,
ControllerBase.Forbid(AuthenticationProperties, params string[]) ,
ControllerBase.SignIn(ClaimsPrincipal) , ControllerBase.SignIn(ClaimsPrincipal, string) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties, string) ,
ControllerBase.SignOut() , ControllerBase.SignOut(AuthenticationProperties) ,
ControllerBase.SignOut(params string[]) ,
ControllerBase.SignOut(AuthenticationProperties, params string[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync(object, Type, string) ,
ControllerBase.TryUpdateModelAsync(object, Type, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryValidateModel(object) , ControllerBase.TryValidateModel(object, string) ,
ControllerBase.HttpContext , ControllerBase.Request , ControllerBase.Response ,
ControllerBase.RouteData , ControllerBase.ModelState , ControllerBase.ControllerContext ,
ControllerBase.MetadataProvider , ControllerBase.ModelBinderFactory , ControllerBase.Url ,
ControllerBase.ObjectValidator , ControllerBase.ProblemDetailsFactory ,
ControllerBase.User , ControllerBase.Empty , object.Equals(object) ,
object.Equals(object, object) , object.GetHashCode() , object.GetType() ,
object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

AuthController(IAuthService)

Handles authentication operations including user login and registration.

```
public AuthController(IAuthService authService)
```

Parameters

`authService` IAuthService

Methods

Login(LoginRequest)

Authenticates a user and returns an access token.

```
[HttpPost("login")]
public Task<IActionResult> Login(LoginRequest dto)
```

Parameters

`dto` LoginRequest

The login credentials containing email and password.

Returns

Task<IActionResult>

OkObjectResult with a FractPal.Model.DTO.Auth.LoginResponse on success; BadRequestObjectResult if the request body is invalid; UnauthorizedObjectResult if credentials are incorrect.

Register(RegistrationRequest)

Registers a new user account.

```
[HttpPost("register")]
public Task<IActionResult> Register(RegistrationRequest dto)
```

Parameters

`dto` RegistrationRequest

The registration details including username, email and password.

Returns

Task<IActionResult>

OkObjectResult with a FractPal.Model.DTO.Auth.RegistrationResponse on success; BadRequestObjectResult if the request body is invalid or the username/email is already taken.

Class CommentsController

Namespace: FractPal.API.Controllers

Assembly: FractPal.API.dll

Handles comment CRUD operations on posts. All endpoints require an authenticated user.

```
[Route("api/[controller]")]
[ApiController]
[Authorize]
public class CommentsController : ControllerBase
```

Inheritance

object ← ControllerBase ← CommentsController

Inherited Members

ControllerBase.StatusCode(int) , ControllerBase.StatusCode(int, object) ,
ControllerBase.Content(string) , ControllerBase.Content(string, string) ,
ControllerBase.Content(string, string, Encoding) ,
ControllerBase.Content(string, MediaTypeHeaderValue) , ControllerBase.NoContent() ,
ControllerBase.Ok() , ControllerBase.Ok(object) , ControllerBase.Redirect(string) ,
ControllerBase.RedirectPermanent(string) , ControllerBase.RedirectPreserveMethod(string) ,
ControllerBase.RedirectPermanentPreserveMethod(string) ,
ControllerBase.LocalRedirect(string) , ControllerBase.LocalRedirectPermanent(string) ,
ControllerBase.LocalRedirectPreserveMethod(string) ,
ControllerBase.LocalRedirectPermanentPreserveMethod(string) ,
ControllerBase.RedirectToAction() , ControllerBase.RedirectToAction(string) ,
ControllerBase.RedirectToAction(string, object) ,
ControllerBase.RedirectToAction(string, string) ,
ControllerBase.RedirectToAction(string, string, object) ,
ControllerBase.RedirectToAction(string, string, string) ,
ControllerBase.RedirectToAction(string, string, object, string) ,
ControllerBase.RedirectToActionPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToActionPermanent(string) ,
ControllerBase.RedirectToActionPermanent(string, object) ,
ControllerBase.RedirectToActionPermanent(string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, object) ,
ControllerBase.RedirectToActionPermanent(string, string, object, string) ,
ControllerBase.RedirectToActionPermanentPreserveMethod(string, string, object, string) ,

ControllerBase.RedirectToRoute(string) , ControllerBase.RedirectToRoute(object) ,
ControllerBase.RedirectToRoute(string, object) ,
ControllerBase.RedirectToRoute(string, string) ,
ControllerBase.RedirectToRoute(string, object, string) ,
ControllerBase.RedirectToRoutePreserveMethod(string, object, string) ,
ControllerBase.RedirectToRoutePermanent(string) ,
ControllerBase.RedirectToRoutePermanent(object) ,
ControllerBase.RedirectToRoutePermanent(string, object) ,
ControllerBase.RedirectToRoutePermanent(string, string) ,
ControllerBase.RedirectToRoutePermanent(string, object, string) ,
ControllerBase.RedirectToRoutePermanentPreserveMethod(string, object, string) ,
ControllerBase.RedirectToPage(string) , ControllerBase.RedirectToPage(string, object) ,
ControllerBase.RedirectToPage(string, string) ,
ControllerBase.RedirectToPage(string, string, object) ,
ControllerBase.RedirectToPage(string, string, string) ,
ControllerBase.RedirectToPage(string, string, object, string) ,
ControllerBase.RedirectToPagePermanent(string) ,
ControllerBase.RedirectToPagePermanent(string, object) ,
ControllerBase.RedirectToPagePermanent(string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, object, string) ,
ControllerBase.RedirectToPagePreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToPagePermanentPreserveMethod(string, string, object, string) ,
ControllerBase.File(byte[], string) , ControllerBase.File(byte[], string, bool) ,
ControllerBase.File(byte[], string, string) , ControllerBase.File(byte[], string, string, bool) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string) , ControllerBase.File(Stream, string, bool) ,
ControllerBase.File(Stream, string, string) , ControllerBase.File(Stream, string, string, bool) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string) , ControllerBase.File(string, string, bool) ,
ControllerBase.File(string, string, string) , ControllerBase.File(string, string, string, bool) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,

ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string) , ControllerBase.PhysicalFile(string, string, bool) ,
ControllerBase.PhysicalFile(string, string, string) ,
ControllerBase.PhysicalFile(string, string, string, bool) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.Unauthorized() , ControllerBase.Unauthorized(object) ,
ControllerBase.NotFound() , ControllerBase.NotFound(object) , ControllerBase.BadRequest() ,
ControllerBase.BadRequest(object) , ControllerBase.BadRequest(ModelStateDictionary) ,
ControllerBase.UnprocessableEntity() , ControllerBase.UnprocessableEntity(object) ,
ControllerBase.UnprocessableEntity(ModelStateDictionary) , ControllerBase.Conflict() ,
ControllerBase.Conflict(object) , ControllerBase.Conflict(ModelStateDictionary) ,
ControllerBase.Problem(string, string, int?, string, string) ,
ControllerBase.Problem(string, string, int?, string, string, IDictionary<string, object>) ,
ControllerBase.ValidationProblem(ValidationProblemDetails) ,
ControllerBase.ValidationProblem(ModelStateDictionary) , ControllerBase.ValidationProblem() ,
ControllerBase.ValidationProblem(string, string, int?, string, string, ModelStateDictionary) ,
ControllerBase.ValidationProblem(string, string, int?, string, string, ModelStateDictionary,
IDictionary<string, object>) ,
ControllerBase.Created() , ControllerBase.Created(string, object) ,
ControllerBase.Created(Uri, object) , ControllerBase.CreatedAtAction(string, object) ,
ControllerBase.CreatedAtAction(string, object, object) ,
ControllerBase.CreatedAtAction(string, string, object, object) ,
ControllerBase.CreatedAtRoute(string, object) , ControllerBase.CreatedAtRoute(object, object) ,
ControllerBase.CreatedAtRoute(string, object, object) , ControllerBase.Accepted() ,
ControllerBase.Accepted(object) , ControllerBase.Accepted(Uri) ,
ControllerBase.Accepted(string) , ControllerBase.Accepted(string, object) ,
ControllerBase.Accepted(Uri, object) , ControllerBase.AcceptedAtAction(string) ,
ControllerBase.AcceptedAtAction(string, string) ,
ControllerBase.AcceptedAtAction(string, object) ,
ControllerBase.AcceptedAtAction(string, string, object) ,
ControllerBase.AcceptedAtAction(string, object, object) ,
ControllerBase.AcceptedAtAction(string, string, object, object) ,
ControllerBase.AcceptedAtRoute(object) , ControllerBase.AcceptedAtRoute(string) ,
ControllerBase.AcceptedAtRoute(string, object) ,
ControllerBase.AcceptedAtRoute(object, object) ,
ControllerBase.AcceptedAtRoute(string, object, object) , ControllerBase.Challenge() ,
ControllerBase.Challenge(params string[]) ,

ControllerBase.Challenge(AuthenticationProperties) ,
ControllerBase.Challenge(AuthenticationProperties, params string[]) , ControllerBase.Forbid() ,
ControllerBase.Forbid(params string[]) , ControllerBase.Forbid(AuthenticationProperties) ,
ControllerBase.Forbid(AuthenticationProperties, params string[]) ,
ControllerBase.SignIn(ClaimsPrincipal) , ControllerBase.SignIn(ClaimsPrincipal, string) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties, string) ,
ControllerBase.SignOut() , ControllerBase.SignOut(AuthenticationProperties) ,
ControllerBase.SignOut(params string[]) ,
ControllerBase.SignOut(AuthenticationProperties, params string[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync(object, Type, string) ,
ControllerBase.TryUpdateModelAsync(object, Type, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryValidateModel(object) , ControllerBase.TryValidateModel(object, string) ,
ControllerBase.HttpContext , ControllerBase.Request , ControllerBase.Response ,
ControllerBase.RouteData , ControllerBase.ModelState , ControllerBase.ControllerContext ,
ControllerBase.MetadataProvider , ControllerBase.ModelBinderFactory , ControllerBase.Url ,
ControllerBase.ObjectValidator , ControllerBase.ProblemDetailsFactory ,
ControllerBase.User , ControllerBase.Empty , object.Equals(object) ,
object.Equals(object, object) , object.GetHashCode() , object.GetType() ,
object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

CommentsController(ICommentService)

Handles comment CRUD operations on posts. All endpoints require an authenticated user.

```
public CommentsController(ICommentService commentService)
```

Parameters

`commentService` ICommentService

Methods

CreateComment(Guid, CreateCommentRequest)

Creates a new comment on the specified post by the authenticated user.

```
[HttpPost("{postId:guid}")]
public Task<IActionResult> CreateComment(Guid postId, CreateCommentRequest request)
```

Parameters

`postId` Guid

The unique identifier of the post to comment on.

`request` CreateCommentRequest

The comment content.

Returns

Task<IActionResult>

CreatedAtActionResult pointing to GetCommentById(Guid) with the new FractPal.Model.DTO.Comment.CommentDto; BadRequestObjectResult if the request body fails validation; NotFoundObjectResult if no post exists with the given ID; 500 if an unexpected error occurs.

DeleteComment(Guid)

Permanently deletes a comment. Only the comment's author may delete it.

```
[HttpDelete("{id:guid}")]
public Task<IActionResult> DeleteComment(Guid id)
```

Parameters

`id` Guid

The unique identifier of the comment to delete.

Returns

Task<IActionResult>

NoContentResult on successful deletion; NotFoundObjectResult if no comment exists with the given ID; ForbidResult if the authenticated user is not the comment's author; 500 if an unexpected error occurs.

GetCommentById(Guid)

Retrieves a single comment by its unique identifier.

```
[HttpGet("{id:guid}")]
public Task<IActionResult> GetCommentById(Guid id)
```

Parameters

`id` Guid

The unique identifier of the comment.

Returns

Task<IActionResult>

OkObjectResult with a FractPal.Model.DTO.Comment.CommentDto on success; NotFoundObjectResult if no comment exists with the given ID; 500 if an unexpected error occurs.

GetPostComments(Guid)

Retrieves all comments for a given post, ordered by creation time (oldest first).

```
[HttpGet("post/{postId:guid}")]
public Task<IActionResult> GetPostComments(Guid postId)
```

Parameters

`postId` Guid

The unique identifier of the post whose comments to retrieve.

Returns

Task<IActionResult>

OkObjectResult with a list of FractPal.Model.DTO.Comment.CommentDto; 500 if an unexpected error occurs.

UpdateComment(Guid, UpdateCommentRequest)

Updates the content of an existing comment. Only the comment's author may edit it.

```
[HttpPut("{id:guid}")]
public Task<IActionResult> UpdateComment(Guid id, UpdateCommentRequest request)
```

Parameters

`id` Guid

The unique identifier of the comment to update.

`request` UpdateCommentRequest

The updated comment content.

Returns

Task<IActionResult>

OkObjectResult with the updated FractPal.Model.DTO.Comment.CommentDto; BadRequestObjectResult if the request body fails validation; NotFoundObjectResult if no comment exists with the given ID; ForbidResult if the authenticated user is not the comment's author; 500 if an unexpected error occurs.

Class FractalController

Namespace: FractPal.API.Controllers

Assembly: FractPal.API.dll

Handles all fractal CRUD operations, feed retrieval, forking, and social interactions (likes). All endpoints require an authenticated user.

```
[Route("api/[controller]")]
[ApiController]
[Authorize]
public class FractalController : ControllerBase
```

Inheritance

object ← ControllerBase ← FractalController

Inherited Members

ControllerBase.StatusCode(int) , ControllerBase.StatusCode(int, object) ,
ControllerBase.Content(string) , ControllerBase.Content(string, string) ,
ControllerBase.Content(string, string, Encoding) ,
ControllerBase.Content(string, MediaTypeHeaderValue) , ControllerBase.NoContent() ,
ControllerBase.Ok() , ControllerBase.Ok(object) , ControllerBase.Redirect(string) ,
ControllerBase.RedirectPermanent(string) , ControllerBase.RedirectPreserveMethod(string) ,
ControllerBase.RedirectPermanentPreserveMethod(string) ,
ControllerBase.LocalRedirect(string) , ControllerBase.LocalRedirectPermanent(string) ,
ControllerBase.LocalRedirectPreserveMethod(string) ,
ControllerBase.LocalRedirectPermanentPreserveMethod(string) ,
ControllerBase.RedirectToAction() , ControllerBase.RedirectToAction(string) ,
ControllerBase.RedirectToAction(string, object) ,
ControllerBase.RedirectToAction(string, string) ,
ControllerBase.RedirectToAction(string, string, object) ,
ControllerBase.RedirectToAction(string, string, string) ,
ControllerBase.RedirectToAction(string, string, object, string) ,
ControllerBase.RedirectToActionPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToActionPermanent(string) ,
ControllerBase.RedirectToActionPermanent(string, object) ,
ControllerBase.RedirectToActionPermanent(string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, object) ,
ControllerBase.RedirectToActionPermanent(string, string, object, string) ,

ControllerBase.RedirectToActionPermanentPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToRoute(string) , ControllerBase.RedirectToRoute(object) ,
ControllerBase.RedirectToRoute(string, object) ,
ControllerBase.RedirectToRoute(string, string) ,
ControllerBase.RedirectToRoute(string, object, string) ,
ControllerBase.RedirectToRoutePreserveMethod(string, object, string) ,
ControllerBase.RedirectToRoutePermanent(string) ,
ControllerBase.RedirectToRoutePermanent(object) ,
ControllerBase.RedirectToRoutePermanent(string, object) ,
ControllerBase.RedirectToRoutePermanent(string, string) ,
ControllerBase.RedirectToRoutePermanent(string, object, string) ,
ControllerBase.RedirectToRoutePermanentPreserveMethod(string, object, string) ,
ControllerBase.RedirectToPage(string) , ControllerBase.RedirectToPage(string, object) ,
ControllerBase.RedirectToPage(string, string) ,
ControllerBase.RedirectToPage(string, string, object) ,
ControllerBase.RedirectToPage(string, string, string) ,
ControllerBase.RedirectToPage(string, string, object, string) ,
ControllerBase.RedirectToPagePermanent(string) ,
ControllerBase.RedirectToPagePermanent(string, object) ,
ControllerBase.RedirectToPagePermanent(string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, object, string) ,
ControllerBase.RedirectToPagePreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToPagePermanentPreserveMethod(string, string, object, string) ,
ControllerBase.File(byte[], string) , ControllerBase.File(byte[], string, bool) ,
ControllerBase.File(byte[], string, string) , ControllerBase.File(byte[], string, string, bool) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string) , ControllerBase.File(Stream, string, bool) ,
ControllerBase.File(Stream, string, string) , ControllerBase.File(Stream, string, string, bool) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string) , ControllerBase.File(string, string, bool) ,
ControllerBase.File(string, string, string) , ControllerBase.File(string, string, string, bool) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,

ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string) , ControllerBase.PhysicalFile(string, string, bool) ,
ControllerBase.PhysicalFile(string, string, string) ,
ControllerBase.PhysicalFile(string, string, string, bool) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.Unauthorized() , ControllerBase.Unauthorized(object) ,
ControllerBase.NotFound() , ControllerBase.NotFound(object) , ControllerBase.BadRequest() ,
ControllerBase.BadRequest(object) , ControllerBase.BadRequest(ModelStateDictionary) ,
ControllerBase.UnprocessableEntity() , ControllerBase.UnprocessableEntity(object) ,
ControllerBase.UnprocessableEntity(ModelStateDictionary) , ControllerBase.Conflict() ,
ControllerBase.Conflict(object) , ControllerBase.Conflict(ModelStateDictionary) ,
ControllerBase.Problem(string, string, int?, string, string) ,
ControllerBase.Problem(string, string, int?, string, string, IDictionary<string, object>) ,
ControllerBase.ValidationProblem(ValidationProblemDetails) ,
ControllerBase.ValidationProblem(ModelStateDictionary) , ControllerBase.ValidationProblem() ,
ControllerBase.ValidationProblem(string, string, int?, string, string, ModelStateDictionary) ,
ControllerBase.ValidationProblem(string, string, int?, string, string, ModelStateDictionary,
IDictionary<string, object>) ,
ControllerBase.Created() , ControllerBase.Created(string, object) ,
ControllerBase.Created(Uri, object) , ControllerBase.CreatedAtAction(string, object) ,
ControllerBase.CreatedAtAction(string, object, object) ,
ControllerBase.CreatedAtAction(string, string, object, object) ,
ControllerBase.CreatedAtRoute(string, object) , ControllerBase.CreatedAtRoute(object, object) ,
ControllerBase.CreatedAtRoute(string, object, object) , ControllerBase.Accepted() ,
ControllerBase.Accepted(object) , ControllerBase.Accepted(Uri) ,
ControllerBase.Accepted(string) , ControllerBase.Accepted(string, object) ,
ControllerBase.Accepted(Uri, object) , ControllerBase.AcceptedAtAction(string) ,
ControllerBase.AcceptedAtAction(string, string) ,
ControllerBase.AcceptedAtAction(string, object) ,
ControllerBase.AcceptedAtAction(string, string, object) ,
ControllerBase.AcceptedAtAction(string, object, object) ,
ControllerBase.AcceptedAtAction(string, string, object, object) ,
ControllerBase.AcceptedAtRoute(object) , ControllerBase.AcceptedAtRoute(string) ,
ControllerBase.AcceptedAtRoute(string, object) ,
ControllerBase.AcceptedAtRoute(object, object) ,
ControllerBase.AcceptedAtRoute(string, object, object) , ControllerBase.Challenge() ,

ControllerBase.Challenge(params string[]) ,
ControllerBase.Challenge(AuthenticationProperties) ,
ControllerBase.Challenge(AuthenticationProperties, params string[]) , ControllerBase.Forbid() ,
ControllerBase.Forbid(params string[]) , ControllerBase.Forbid(AuthenticationProperties) ,
ControllerBase.Forbid(AuthenticationProperties, params string[]) ,
ControllerBase.SignIn(ClaimsPrincipal) , ControllerBase.SignIn(ClaimsPrincipal, string) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties, string) ,
ControllerBase.SignOut() , ControllerBase.SignOut(AuthenticationProperties) ,
ControllerBase.SignOut(params string[]) ,
ControllerBase.SignOut(AuthenticationProperties, params string[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync(object, Type, string) ,
ControllerBase.TryUpdateModelAsync(object, Type, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryValidateModel(object) , ControllerBase.TryValidateModel(object, string) ,
ControllerBase.HttpContext , ControllerBase.Request , ControllerBase.Response ,
ControllerBase.RouteData , ControllerBase.ModelState , ControllerBase.ControllerContext ,
ControllerBase.MetadataProvider , ControllerBase.ModelBinderFactory , ControllerBase.Url ,
ControllerBase.ObjectValidator , ControllerBase.ProblemDetailsFactory ,
ControllerBase.User , ControllerBase.Empty , object.Equals(object) ,
object.Equals(object, object) , object.GetHashCode() , object.GetType() ,
object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

FractalController(IFractalService, ILikeService)

Handles all fractal CRUD operations, feed retrieval, forking, and social interactions (likes). All endpoints require an authenticated user.

```
public FractalController(IFractalService fractalService, ILikeService likeService)
```

Parameters

`fractalService` IFractalService

`likeService` ILikeService

Methods

CreateFractal(CreateFractalRequest)

Creates a new fractal owned by the authenticated user. The fractal starts as a draft and must be explicitly published via PublishFractal.

```
[HttpPost]
public Task<IActionResult> CreateFractal(CreateFractalRequest request)
```

Parameters

`request` CreateFractalRequest

The fractal configuration data including L-system rules and optional thumbnail.

Returns

Task<IActionResult>

CreatedAtActionResult pointing to GetFractalById(Guid) with the new FractPal.Model.DTO.Fractal.FractalDto; BadRequestObjectResult if the request body fails validation; 500 if an unexpected error occurs.

DeleteFractal(Guid)

Permanently deletes a fractal. Only the owner may delete their own fractal.

```
[HttpDelete("{id}")]
public Task<IActionResult> DeleteFractal(Guid id)
```

Parameters

`id` Guid

The unique identifier of the fractal to delete.

Returns

Task<IActionResult>

NoContentResult on successful deletion; NotFoundObjectResult if no fractal exists with the given ID; ForbidResult if the authenticated user is not the owner; 500 if an unexpected error occurs.

ForkFractal(Guid)

Creates a copy (fork) of an existing published fractal under the authenticated user's account. The forked fractal starts as a draft.

```
[HttpPost("{id}/fork")]
public Task<IActionResult> ForkFractal(Guid id)
```

Parameters

`id` Guid

The unique identifier of the fractal to fork.

Returns

Task<IActionResult>

CreatedAtActionResult pointing to GetFractalById(Guid) with the new forked FractPal.Model.DTO.Fractal.FractalDto; NotFoundObjectResult if no fractal exists with the given ID; 500 if an unexpected error occurs.

GetFeed(int, int)

Returns a paginated feed of all published fractals, ordered by most recently published.

```
[HttpGet("feed")]
public Task<IActionResult> GetFeed(int page = 1, int pageSize = 20)
```

Parameters

`page` int

The 1-based page number. Defaults to 1.

`pageSize` int

The number of fractals per page. Defaults to 20.

Returns

Task<IActionResult>

OkObjectResult with a FractPal.Model.DTO.Fractal.FractalFeedResponse containing the fractals and pagination metadata; 500 if an unexpected error occurs.

GetFractalById(Guid)

Retrieves a single fractal by its unique identifier.

```
[HttpGet("{id}")]
public Task<IActionResult> GetFractalById(Guid id)
```

Parameters

`id` Guid

The unique identifier of the fractal.

Returns

Task<IActionResult>

OkObjectResult with a FractPal.Model.DTO.Fractal.FractalDto on success; NotFoundObjectResult if no fractal exists with the given ID; 500 if an unexpected error occurs.

GetMyFractals()

Returns all fractals (published and draft) belonging to the authenticated user.

```
[HttpGet("mine")]
public Task<IActionResult> GetMyFractals()
```

Returns

Task<IActionResult>

OkObjectResult with a list of FractPal.Model.DTO.Fractal.FractalDto; 500 if an unexpected error occurs.

GetUserFractals(Guid)

Returns all published fractals belonging to a specific user.

```
[HttpGet("user/{userId}")]
public Task<IActionResult> GetUserFractals(Guid userId)
```

Parameters

`userId` Guid

The ID of the user whose published fractals to retrieve.

Returns

Task<IActionResult>

OkObjectResult with a list of FractPal.Model.DTO.Fractal.FractalDto; 500 if an unexpected error occurs.

UpdateFractal(Guid, UpdateFractalRequest)

Updates an existing fractal. If the authenticated user is not the owner, a forked copy is created instead and the original is left unchanged.

```
[HttpPut("{id}")]
public Task<IActionResult> UpdateFractal(Guid id, UpdateFractalRequest request)
```

Parameters

`id` Guid

The unique identifier of the fractal to update.

`request` UpdateFractalRequest

The updated fractal configuration data.

Returns

Task<IActionResult>

OkObjectResult with the updated or forked FractPal.Model.DTO.Fractal.FractalDto; BadRequestObjectResult if the request body fails validation; NotFoundObjectResult if no fractal exists with the given ID; ForbidResult if the user lacks permission (owner check is done in service); 500 if an unexpected error occurs.

Class PostsController

Namespace: FractPal.API.Controllers

Assembly: FractPal.API.dll

Handles post lifecycle operations including feed retrieval, publishing and unpublishing fractals as posts, and social interactions (likes). All endpoints require an authenticated user.

```
[Route("api/[controller]")]
[ApiController]
[Authorize]
public class PostsController : ControllerBase
```

Inheritance

object ← ControllerBase ← PostsController

Inherited Members

ControllerBase.StatusCode(int) , ControllerBase.StatusCode(int, object) ,
ControllerBase.Content(string) , ControllerBase.Content(string, string) ,
ControllerBase.Content(string, string, Encoding) ,
ControllerBase.Content(string, MediaTypeHeaderValue) , ControllerBase.NoContent() ,
ControllerBase.Ok() , ControllerBase.Ok(object) , ControllerBase.Redirect(string) ,
ControllerBase.RedirectPermanent(string) , ControllerBase.RedirectPreserveMethod(string) ,
ControllerBase.RedirectPermanentPreserveMethod(string) ,
ControllerBase.LocalRedirect(string) , ControllerBase.LocalRedirectPermanent(string) ,
ControllerBase.LocalRedirectPreserveMethod(string) ,
ControllerBase.LocalRedirectPermanentPreserveMethod(string) ,
ControllerBase.RedirectToAction() , ControllerBase.RedirectToAction(string) ,
ControllerBase.RedirectToAction(string, object) ,
ControllerBase.RedirectToAction(string, string) ,
ControllerBase.RedirectToAction(string, string, object) ,
ControllerBase.RedirectToAction(string, string, string) ,
ControllerBase.RedirectToAction(string, string, object, string) ,
ControllerBase.RedirectToActionPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToActionPermanent(string) ,
ControllerBase.RedirectToActionPermanent(string, object) ,
ControllerBase.RedirectToActionPermanent(string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, object) ,
ControllerBase.RedirectToActionPermanent(string, string, object, string) ,

ControllerBase.RedirectToActionPermanentPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToRoute(string) , ControllerBase.RedirectToRoute(object) ,
ControllerBase.RedirectToRoute(string, object) ,
ControllerBase.RedirectToRoute(string, string) ,
ControllerBase.RedirectToRoute(string, object, string) ,
ControllerBase.RedirectToRoutePreserveMethod(string, object, string) ,
ControllerBase.RedirectToRoutePermanent(string) ,
ControllerBase.RedirectToRoutePermanent(object) ,
ControllerBase.RedirectToRoutePermanent(string, object) ,
ControllerBase.RedirectToRoutePermanent(string, string) ,
ControllerBase.RedirectToRoutePermanent(string, object, string) ,
ControllerBase.RedirectToRoutePermanentPreserveMethod(string, object, string) ,
ControllerBase.RedirectToPage(string) , ControllerBase.RedirectToPage(string, object) ,
ControllerBase.RedirectToPage(string, string) ,
ControllerBase.RedirectToPage(string, string, object) ,
ControllerBase.RedirectToPage(string, string, string) ,
ControllerBase.RedirectToPage(string, string, object, string) ,
ControllerBase.RedirectToPagePermanent(string) ,
ControllerBase.RedirectToPagePermanent(string, object) ,
ControllerBase.RedirectToPagePermanent(string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, object, string) ,
ControllerBase.RedirectToPagePreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToPagePermanentPreserveMethod(string, string, object, string) ,
ControllerBase.File(byte[], string) , ControllerBase.File(byte[], string, bool) ,
ControllerBase.File(byte[], string, string) , ControllerBase.File(byte[], string, string, bool) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string) , ControllerBase.File(Stream, string, bool) ,
ControllerBase.File(Stream, string, string) , ControllerBase.File(Stream, string, string, bool) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string) , ControllerBase.File(string, string, bool) ,
ControllerBase.File(string, string, string) , ControllerBase.File(string, string, string, bool) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,

ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string) , ControllerBase.PhysicalFile(string, string, bool) ,
ControllerBase.PhysicalFile(string, string, string) ,
ControllerBase.PhysicalFile(string, string, string, bool) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.Unauthorized() , ControllerBase.Unauthorized(object) ,
ControllerBase.NotFound() , ControllerBase.NotFound(object) , ControllerBase.BadRequest() ,
ControllerBase.BadRequest(object) , ControllerBase.BadRequest(ModelStateDictionary) ,
ControllerBase.UnprocessableEntity() , ControllerBase.UnprocessableEntity(object) ,
ControllerBase.UnprocessableEntity(ModelStateDictionary) , ControllerBase.Conflict() ,
ControllerBase.Conflict(object) , ControllerBase.Conflict(ModelStateDictionary) ,
ControllerBase.Problem(string, string, int?, string, string) ,
ControllerBase.Problem(string, string, int?, string, string, IDictionary<string, object>) ,
ControllerBase.ValidationProblem(ValidationProblemDetails) ,
ControllerBase.ValidationProblem(ModelStateDictionary) , ControllerBase.ValidationProblem() ,
ControllerBase.ValidationProblem(string, string, int?, string, string, ModelStateDictionary) ,
ControllerBase.ValidationProblem(string, string, int?, string, string, ModelStateDictionary,
IDictionary<string, object>) ,
ControllerBase.Created() , ControllerBase.Created(string, object) ,
ControllerBase.Created(Uri, object) , ControllerBase.CreatedAtAction(string, object) ,
ControllerBase.CreatedAtAction(string, object, object) ,
ControllerBase.CreatedAtAction(string, string, object, object) ,
ControllerBase.CreatedAtRoute(string, object) , ControllerBase.CreatedAtRoute(object, object) ,
ControllerBase.CreatedAtRoute(string, object, object) , ControllerBase.Accepted() ,
ControllerBase.Accepted(object) , ControllerBase.Accepted(Uri) ,
ControllerBase.Accepted(string) , ControllerBase.Accepted(string, object) ,
ControllerBase.Accepted(Uri, object) , ControllerBase.AcceptedAtAction(string) ,
ControllerBase.AcceptedAtAction(string, string) ,
ControllerBase.AcceptedAtAction(string, object) ,
ControllerBase.AcceptedAtAction(string, string, object) ,
ControllerBase.AcceptedAtAction(string, object, object) ,
ControllerBase.AcceptedAtAction(string, string, object, object) ,
ControllerBase.AcceptedAtRoute(object) , ControllerBase.AcceptedAtRoute(string) ,
ControllerBase.AcceptedAtRoute(string, object) ,
ControllerBase.AcceptedAtRoute(object, object) ,
ControllerBase.AcceptedAtRoute(string, object, object) , ControllerBase.Challenge() ,

ControllerBase.Challenge(params string[]) ,
ControllerBase.Challenge(AuthenticationProperties) ,
ControllerBase.Challenge(AuthenticationProperties, params string[]) , ControllerBase.Forbid() ,
ControllerBase.Forbid(params string[]) , ControllerBase.Forbid(AuthenticationProperties) ,
ControllerBase.Forbid(AuthenticationProperties, params string[]) ,
ControllerBase.SignIn(ClaimsPrincipal) , ControllerBase.SignIn(ClaimsPrincipal, string) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties, string) ,
ControllerBase.SignOut() , ControllerBase.SignOut(AuthenticationProperties) ,
ControllerBase.SignOut(params string[]) ,
ControllerBase.SignOut(AuthenticationProperties, params string[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync(object, Type, string) ,
ControllerBase.TryUpdateModelAsync(object, Type, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryValidateModel(object) , ControllerBase.TryValidateModel(object, string) ,
ControllerBase.HttpContext , ControllerBase.Request , ControllerBase.Response ,
ControllerBase.RouteData , ControllerBase.ModelState , ControllerBase.ControllerContext ,
ControllerBase.MetadataProvider , ControllerBase.ModelBinderFactory , ControllerBase.Url ,
ControllerBase.ObjectValidator , ControllerBase.ProblemDetailsFactory ,
ControllerBase.User , ControllerBase.Empty , object.Equals(object) ,
object.Equals(object, object) , object.GetHashCode() , object.GetType() ,
object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

PostsController(IPostService, ILikeService)

Handles post lifecycle operations including feed retrieval, publishing and unpublishing fractals as posts, and social interactions (likes). All endpoints require an authenticated user.

```
public PostsController(IPostService postService, ILikeService likeService)
```

Parameters

`postService` IPostService

`likeService` ILikeService

Methods

DeletePost(Guid)

Permanently deletes a post. Admins may delete any post; regular users may only delete their own.

```
[HttpDelete("{id:guid}")]
public Task<IActionResult> DeletePost(Guid id)
```

Parameters

`id` Guid

The unique identifier of the post to delete.

Returns

Task<IActionResult>

NoContentResult on successful deletion; NotFoundObjectResult if no post exists with the given ID; ForbidResult if the authenticated user is not the post's author and is not an admin; 500 if an unexpected error occurs.

GetFeed(int, int)

Returns a paginated feed of all posts, ordered by most recently created.

```
[HttpGet("feed")]
public Task<IActionResult> GetFeed(int page = 1, int pageSize = 20)
```

Parameters

`page` int

The 1-based page number. Defaults to 1.

`pageSize` int

The number of posts per page. Defaults to 20.

Returns

Task<IActionResult>

OkObjectResult with a FractPal.Model.DTO.Post.PostFeedResponse containing the posts and pagination metadata; 500 if an unexpected error occurs.

GetMyPosts()

Returns all posts belonging to the authenticated user.

```
[HttpGet("my")]
public Task<IActionResult> GetMyPosts()
```

Returns

Task<IActionResult>

OkObjectResult with a list of FractPal.Model.DTO.Post.PostDto; 500 if an unexpected error occurs.

GetPostById(Guid)

Retrieves a single post by its unique identifier.

```
[HttpGet("{id:guid}")]
public Task<IActionResult> GetPostById(Guid id)
```

Parameters

`id` Guid

The unique identifier of the post.

Returns

Task<IActionResult>

OkObjectResult with a FractPal.Model.DTO.Post.PostDto on success; NotFoundObjectResult if no post exists with the given ID; 500 if an unexpected error occurs.

GetUserPosts(Guid)

Returns all posts belonging to a specific user.

```
[HttpGet("user/{userId:guid}")]
public Task<IActionResult> GetUserPosts(Guid userId)
```

Parameters

`userId` Guid

The ID of the user whose posts to retrieve.

Returns

Task<IActionResult>

OkObjectResult with a list of FractPal.Model.DTO.Post.PostDto; 500 if an unexpected error occurs.

PublishFractal(Guid, CreatePostRequest)

Publishes a fractal as a post. Only the fractal's owner may publish it.

```
[HttpPost("publish/{fractalId:guid}")]
public Task<IActionResult> PublishFractal(Guid fractalId, CreatePostRequest request)
```

Parameters

`fractalId` Guid

The unique identifier of the fractal to publish.

`request` CreatePostRequest

The post metadata including name and description.

Returns

Task<IActionResult>

CreatedAtActionResult pointing to GetPostById(Guid) with the new FractPal.Model.DTO.Post.PostDto; BadRequestObjectResult if the request body fails validation; NotFoundObjectResult if no fractal exists with the given ID; ForbidResult if the authenticated user is not the fractal's owner; 500 if an unexpected error occurs.

ToggleLikePost(Guid)

Toggles the like state on a post for the authenticated user. Resolves to the underlying fractal's like collection. Calling this when already liked will unlike, and vice versa.

```
[HttpPost("{id:guid}/like")]
public Task<IActionResult> ToggleLikePost(Guid id)
```

Parameters

`id` Guid

The unique identifier of the post to like or unlike.

Returns

Task<IActionResult>

OkObjectResult with an `isLiked` boolean indicating the new state; NotFoundObjectResult if the post or its underlying fractal is not found; 500 if an unexpected error occurs.

UnpublishFractal(Guid)

Removes the published post associated with a fractal, effectively unpublishing it. Only the post's author may unpublish it.

```
[HttpPost("unpublish/{fractalId:guid}")]
public Task<IActionResult> UnpublishFractal(Guid fractalId)
```

Parameters

`fractalId` Guid

The unique identifier of the fractal whose post to remove.

Returns

Task<IActionResult>

NoContentResult on successful removal; NotFoundObjectResult if no matching post exists; ForbidResult if the authenticated user is not the post's author; 500 if an unexpected error occurs.

UpdatePost(Guid, UpdatePostRequest)

Updates the name and description of an existing post. Only the post's author may update it.

```
[HttpPut("{id:guid}")]
public Task<IActionResult> UpdatePost(Guid id, UpdatePostRequest request)
```

Parameters

`id` Guid

The unique identifier of the post to update.

`request` UpdatePostRequest

The updated post metadata.

Returns

Task<IActionResult>

OkObjectResult with the updated FractPal.Model.DTO.Post.PostDto; BadRequestObjectResult if the request body fails validation; NotFoundObjectResult if no post exists with the given ID; ForbidResult if the authenticated user is not the post's author; 500 if an unexpected error occurs.

Class UserController

Namespace: FractPal.API.Controllers

Assembly: FractPal.API.dll

Handles user profile and social operations such as viewing profiles, updating bio, following/unfollowing users, and searching for users.

```
[Route("api/[controller]")]
[ApiController]
[Authorize]
public class UserController : ControllerBase
```

Inheritance

object ← ControllerBase ← UserController

Inherited Members

ControllerBase.StatusCode(int) , ControllerBase.StatusCode(int, object) ,
ControllerBase.Content(string) , ControllerBase.Content(string, string) ,
ControllerBase.Content(string, string, Encoding) ,
ControllerBase.Content(string, MediaTypeHeaderValue) , ControllerBase.NoContent() ,
ControllerBase.Ok() , ControllerBase.Ok(object) , ControllerBase.Redirect(string) ,
ControllerBase.RedirectPermanent(string) , ControllerBase.RedirectPreserveMethod(string) ,
ControllerBase.RedirectPermanentPreserveMethod(string) ,
ControllerBase.LocalRedirect(string) , ControllerBase.LocalRedirectPermanent(string) ,
ControllerBase.LocalRedirectPreserveMethod(string) ,
ControllerBase.LocalRedirectPermanentPreserveMethod(string) ,
ControllerBase.RedirectToAction() , ControllerBase.RedirectToAction(string) ,
ControllerBase.RedirectToAction(string, object) ,
ControllerBase.RedirectToAction(string, string) ,
ControllerBase.RedirectToAction(string, string, object) ,
ControllerBase.RedirectToAction(string, string, string) ,
ControllerBase.RedirectToAction(string, string, object, string) ,
ControllerBase.RedirectToActionPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToActionPermanent(string) ,
ControllerBase.RedirectToActionPermanent(string, object) ,
ControllerBase.RedirectToActionPermanent(string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, object) ,
ControllerBase.RedirectToActionPermanent(string, string, object, string) ,

ControllerBase.RedirectToActionPermanentPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToRoute(string) , ControllerBase.RedirectToRoute(object) ,
ControllerBase.RedirectToRoute(string, object) ,
ControllerBase.RedirectToRoute(string, string) ,
ControllerBase.RedirectToRoute(string, object, string) ,
ControllerBase.RedirectToRoutePreserveMethod(string, object, string) ,
ControllerBase.RedirectToRoutePermanent(string) ,
ControllerBase.RedirectToRoutePermanent(object) ,
ControllerBase.RedirectToRoutePermanent(string, object) ,
ControllerBase.RedirectToRoutePermanent(string, string) ,
ControllerBase.RedirectToRoutePermanent(string, object, string) ,
ControllerBase.RedirectToRoutePermanentPreserveMethod(string, object, string) ,
ControllerBase.RedirectToPage(string) , ControllerBase.RedirectToPage(string, object) ,
ControllerBase.RedirectToPage(string, string) ,
ControllerBase.RedirectToPage(string, string, object) ,
ControllerBase.RedirectToPage(string, string, string) ,
ControllerBase.RedirectToPage(string, string, object, string) ,
ControllerBase.RedirectToPagePermanent(string) ,
ControllerBase.RedirectToPagePermanent(string, object) ,
ControllerBase.RedirectToPagePermanent(string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, object, string) ,
ControllerBase.RedirectToPagePreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToPagePermanentPreserveMethod(string, string, object, string) ,
ControllerBase.File(byte[], string) , ControllerBase.File(byte[], string, bool) ,
ControllerBase.File(byte[], string, string) , ControllerBase.File(byte[], string, string, bool) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string) , ControllerBase.File(Stream, string, bool) ,
ControllerBase.File(Stream, string, string) , ControllerBase.File(Stream, string, string, bool) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string) , ControllerBase.File(string, string, bool) ,
ControllerBase.File(string, string, string) , ControllerBase.File(string, string, string, bool) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,

ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string) , ControllerBase.PhysicalFile(string, string, bool) ,
ControllerBase.PhysicalFile(string, string, string) ,
ControllerBase.PhysicalFile(string, string, string, bool) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.Unauthorized() , ControllerBase.Unauthorized(object) ,
ControllerBase.NotFound() , ControllerBase.NotFound(object) , ControllerBase.BadRequest() ,
ControllerBase.BadRequest(object) , ControllerBase.BadRequest(ModelStateDictionary) ,
ControllerBase.UnprocessableEntity() , ControllerBase.UnprocessableEntity(object) ,
ControllerBase.UnprocessableEntity(ModelStateDictionary) , ControllerBase.Conflict() ,
ControllerBase.Conflict(object) , ControllerBase.Conflict(ModelStateDictionary) ,
ControllerBase.Problem(string, string, int?, string, string) ,
ControllerBase.Problem(string, string, int?, string, string, IDictionary<string, object>) ,
ControllerBase.ValidationProblem(ValidationProblemDetails) ,
ControllerBase.ValidationProblem(ModelStateDictionary) , ControllerBase.ValidationProblem() ,
ControllerBase.ValidationProblem(string, string, int?, string, string, ModelStateDictionary) ,
ControllerBase.ValidationProblem(string, string, int?, string, string, ModelStateDictionary,
IDictionary<string, object>) ,
ControllerBase.Created() , ControllerBase.Created(string, object) ,
ControllerBase.Created(Uri, object) , ControllerBase.CreatedAtAction(string, object) ,
ControllerBase.CreatedAtAction(string, object, object) ,
ControllerBase.CreatedAtAction(string, string, object, object) ,
ControllerBase.CreatedAtRoute(string, object) , ControllerBase.CreatedAtRoute(object, object) ,
ControllerBase.CreatedAtRoute(string, object, object) , ControllerBase.Accepted() ,
ControllerBase.Accepted(object) , ControllerBase.Accepted(Uri) ,
ControllerBase.Accepted(string) , ControllerBase.Accepted(string, object) ,
ControllerBase.Accepted(Uri, object) , ControllerBase.AcceptedAtAction(string) ,
ControllerBase.AcceptedAtAction(string, string) ,
ControllerBase.AcceptedAtAction(string, object) ,
ControllerBase.AcceptedAtAction(string, string, object) ,
ControllerBase.AcceptedAtAction(string, object, object) ,
ControllerBase.AcceptedAtAction(string, string, object, object) ,
ControllerBase.AcceptedAtRoute(object) , ControllerBase.AcceptedAtRoute(string) ,
ControllerBase.AcceptedAtRoute(string, object) ,
ControllerBase.AcceptedAtRoute(object, object) ,
ControllerBase.AcceptedAtRoute(string, object, object) , ControllerBase.Challenge() ,

ControllerBase.Challenge(params string[]) ,
ControllerBase.Challenge(AuthenticationProperties) ,
ControllerBase.Challenge(AuthenticationProperties, params string[]) , ControllerBase.Forbid() ,
ControllerBase.Forbid(params string[]) , ControllerBase.Forbid(AuthenticationProperties) ,
ControllerBase.Forbid(AuthenticationProperties, params string[]) ,
ControllerBase.SignIn(ClaimsPrincipal) , ControllerBase.SignIn(ClaimsPrincipal, string) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties, string) ,
ControllerBase.SignOut() , ControllerBase.SignOut(AuthenticationProperties) ,
ControllerBase.SignOut(params string[]) ,
ControllerBase.SignOut(AuthenticationProperties, params string[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync(object, Type, string) ,
ControllerBase.TryUpdateModelAsync(object, Type, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryValidateModel(object) , ControllerBase.TryValidateModel(object, string) ,
ControllerBase.HttpContext , ControllerBase.Request , ControllerBase.Response ,
ControllerBase.RouteData , ControllerBase.ModelState , ControllerBase.ControllerContext ,
ControllerBase.MetadataProvider , ControllerBase.ModelBinderFactory , ControllerBase.Url ,
ControllerBase.ObjectValidator , ControllerBase.ProblemDetailsFactory ,
ControllerBase.User , ControllerBase.Empty , object.Equals(object) ,
object.Equals(object, object) , object.GetHashCode() , object.GetType() ,
object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

UserController(IUserService)

Handles user profile and social operations such as viewing profiles, updating bio, following/unfollowing users, and searching for users.

```
public UserController(IUserService userService)
```

Parameters

`userService` IUserService

Methods

GetMyProfile()

Retrieves the profile of the currently authenticated user.

```
[HttpGet("profile")]
public Task<IActionResult> GetMyProfile()
```

Returns

Task<IActionResult>

OkObjectResult with a FractPal.Model.DTO.User.UserProfileDto on success; NotFoundObjectResult if the user no longer exists; 500 if an unexpected error occurs.

GetUserProfile(string)

Retrieves the public profile of any user by their ID.

```
[HttpGet("{id}")]
public Task<IActionResult> GetUserProfile(string id)
```

Parameters

`id` string

The ID of the user whose profile to retrieve.

Returns

Task<IActionResult>

OkObjectResult with a FractPal.Model.DTO.User.UserProfileDto on success; NotFoundObjectResult if no user exists with the given ID; 500 if an unexpected error occurs.

SearchUsers(string)

Searches for users whose usernames contain the given query string.

```
[HttpGet("search")]
public Task<IActionResult> SearchUsers(string query)
```

Parameters

`query` string

The search term to match against usernames. Must not be empty.

Returns

Task<IActionResult>

OkObjectResult with a list of FractPal.Model.DTO.User.UserSearchDto on success; BadRequestObjectResult if the query parameter is empty or whitespace; 500 if an unexpected error occurs.

ToggleFollow(string)

Toggles the follow relationship between the authenticated user and the target user. Calling this endpoint when already following will unfollow, and vice versa.

```
[HttpPost("{id}/follow")]
public Task<IActionResult> ToggleFollow(string id)
```

Parameters

`id` string

The ID of the user to follow or unfollow.

Returns

Task<IActionResult>

OkObjectResult with an `isFollowing` boolean indicating the new state; BadRequestObjectResult if attempting to follow oneself; 500 if an unexpected error occurs.

UpdateProfile(UpdateProfileRequest)

Updates the authenticated user's profile information (e.g. bio).

```
[HttpPut("profile")]
public Task<IActionResult> UpdateProfile(UpdateProfileRequest request)
```

Parameters

`request` UpdateProfileRequest

The updated profile data.

Returns

Task<IActionResult>

OkObjectResult with the updated FractPal.Model.DTO.User.UserProfileDto on success; BadRequestObjectResult if the request body fails validation; 500 if an unexpected error occurs.

Namespace FractPal.Data

Classes

Repository<T>

Entity Framework Core implementation of IRepository<T>.

Interfaces

IRepository<T>

Generic repository abstraction providing standard data access operations for any entity type `T`.

Interface IRepository<T>

Namespace: FractPal.Data

Assembly: FractPal.Service.dll

Generic repository abstraction providing standard data access operations for any entity type `T`.

```
public interface IRepository<T> where T : class
```

Type Parameters

`T`

The entity type managed by this repository.

Methods

AddAsync(T)

Persists a new entity to the data store.

```
Task<T> AddAsync(T entity)
```

Parameters

`entity` T

The entity to add.

Returns

Task<T>

The added entity, potentially including database-generated values such as IDs.

CountAsync(Expression<Func<T, bool>>?)

Counts entities, optionally filtered by a predicate.

```
Task<int> CountAsync(Expression<Func<T, bool>>? predicate = null)
```

Parameters

`predicate` Expression<Func<T, bool>>

An optional filter expression. When `null`, all entities are counted.

Returns

Task<int>

The number of matching entities.

DeleteAsync(T)

Removes an entity from the data store.

```
Task DeleteAsync(T entity)
```

Parameters

`entity` T

The entity to delete.

Returns

Task

FindAsync(Expression<Func<T, bool>>)

Retrieves all entities matching the given predicate.

```
Task<IEnumerable<T>> FindAsync(Expression<Func<T, bool>> predicate)
```

Parameters

`predicate` Expression<Func<T, bool>>

A filter expression applied to the entity set.

Returns

Task<IEnumerable<T>>

An enumerable of matching entities.

GetAllAsync()

Retrieves all entities of type `T`.

```
Task<IEnumerable<T>> GetAllAsync()
```

Returns

Task<IEnumerable<T>>

An enumerable of all entities.

GetByIdAsync(string)

Retrieves an entity by its primary key.

```
Task<T?> GetByIdAsync(string id)
```

Parameters

`id` string

The string representation of the primary key.

Returns

Task<T>

The matching entity, or `null` if not found.

UpdateAsync(T)

Updates an existing entity in the data store.

```
Task<T> UpdateAsync(T entity)
```

Parameters

`entity` T

The entity with updated values.

Returns

Task<T>

The updated entity.

Class Repository<T>

Namespace: FractPal.Data

Assembly: FractPal.Service.dll

Entity Framework Core implementation of IRepository<T>.

```
public class Repository<T> : IRepository<T> where T : class
```

Type Parameters

`T`

The entity type managed by this repository.

Inheritance

object ← Repository<T>

Implements

IRepository<T>

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

Repository(ApplicationDbContext)

Entity Framework Core implementation of IRepository<T>.

```
public Repository(ApplicationDbContext context)
```

Parameters

`context` ApplicationDbContext

Properties

Context

The underlying EF Core database context.

```
protected ApplicationDbContext Context { get; }
```

Property Value

ApplicationDbContext

Methods

AddAsync(T)

Persists a new entity to the data store.

```
public virtual Task<T> AddAsync(T entity)
```

Parameters

`entity` T

The entity to add.

Returns

Task<T>

The added entity, potentially including database-generated values such as IDs.

CountAsync(Expression<Func<T, bool>>?)

Counts entities, optionally filtered by a predicate.

```
public virtual Task<int> CountAsync(Expression<Func<T, bool>>? predicate = null)
```

Parameters

`predicate` Expression<Func<T, bool>>

An optional filter expression. When `null`, all entities are counted.

Returns

Task<int>

The number of matching entities.

DeleteAsync(T)

Removes an entity from the data store.

```
public virtual Task DeleteAsync(T entity)
```

Parameters

`entity` T

The entity to delete.

Returns

Task

FindAsync(Expression<Func<T, bool>>)

Retrieves all entities matching the given predicate.

```
public virtual Task<IEnumerable<T>> FindAsync(Expression<Func<T, bool>> predicate)
```

Parameters

`predicate` Expression<Func<T, bool>>

A filter expression applied to the entity set.

Returns

Task<IEnumerable<T>>

An enumerable of matching entities.

GetAllAsync()

Retrieves all entities of type `T`.

```
public virtual Task<IEnumerable<T>> GetAllAsync()
```

Returns

Task<IEnumerable<T>>

An enumerable of all entities.

GetByIdAsync(string)

Retrieves an entity by its primary key.

```
public virtual Task<T?> GetByIdAsync(string id)
```

Parameters

`id` string

The string representation of the primary key.

Returns

Task<T>

The matching entity, or `null` if not found.

UpdateAsync(T)

Updates an existing entity in the data store.

```
public virtual Task<T> UpdateAsync(T entity)
```

Parameters

`entity` T

The entity with updated values.

Returns

Task<T>

The updated entity.

Namespace FractPal.Service.Implementation

Classes

AuthService
: Implements authentication logic including login and new user registration using ASP.NET Core Identity.

CommentService
: Implements comment business logic, including creation, retrieval, editing, and deletion of comments on FractPal posts.

FractalService
: Implements fractal business logic for FractPal, including creation, retrieval, modification, forking, and deletion of fractals.

JwtService
: Generates signed JWT access tokens for authenticated FractPal users, including role claims. Token settings (secret, issuer, audience, expiry) are resolved from environment variables first, falling back to `appsettings.json`.

LikeService
: Implements like/unlike toggle logic for fractals and posts. Likes are stored at the fractal level; post likes resolve to the underlying fractal.

PostService
: Implements post business logic for FractPal, including publishing fractals as posts, feed retrieval, and post lifecycle management.

RefreshTokenService
: Manages refresh token lifecycle: generation, validation, lookup, and revocation. Tokens are cryptographically random, stored in the repository, and expire after 7 days.

UserService
: Implements user profile and social features for FractPal, including profile retrieval and updates, follow/unfollow toggling, and username search.

Class AuthService

Namespace: FractPal.Service.Implementation

Assembly: FractPal.Service.dll

Implements authentication logic including login and new user registration using ASP.NET Core Identity.

```
public class AuthService : IAuthService
```

Inheritance

object ← AuthService

Implements

IAuthService

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

AuthService(UserManager<FractPalUser>, SignInManager<FractPalUser>, IJwtService)

Implements authentication logic including login and new user registration using ASP.NET Core Identity.

```
public AuthService(UserManager<FractPalUser> userManager,
SignInManager<FractPalUser> signInManager, IJwtService jwtService)
```

Parameters

`userManager` UserManager<FractPalUser>

`signInManager` SignInManager<FractPalUser>

`jwtService` IJwtService

Methods

Login(HttpContext, LoginRequest)

Authenticates a user with the provided credentials and issues an access token. Sets any necessary cookies or session state via `context`.

```
public Task<LoginResponse> Login(HttpContext context, LoginRequest request)
```

Parameters

`context` HttpContext

The current HTTP context, used to set authentication cookies if applicable.

`request` LoginRequest

The login credentials containing email and password.

Returns

Task<LoginResponse>

A FractPal.Model.DTO.Auth.LoginResponse containing the access token and user information.

Exceptions

UnauthorizedAccessException

Thrown when the email/password combination is invalid.

Register(RegistrationRequest)

Registers a new user account with the provided details.

```
public Task<RegistrationResponse> Register(RegistrationRequest request)
```

Parameters

`request` RegistrationRequest

The registration data including username, email and password.

Returns

Task<RegistrationResponse>

A FractPal.Model.DTO.Auth.RegistrationResponse confirming the created account.

Exceptions

InvalidOperationException

Thrown when the email or username is already in use.

Class CommentService

Namespace: FractPal.Service.Implementation

Assembly: FractPal.Service.dll

Implements comment business logic, including creation, retrieval, editing, and deletion of comments on FractPal posts.

```
public class CommentService : ICommentService
```

Inheritance

object ← CommentService

Implements

ICommentService

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

CommentService(ApplicationDbContext)

Implements comment business logic, including creation, retrieval, editing, and deletion of comments on FractPal posts.

```
public CommentService(ApplicationDbContext dbContext)
```

Parameters

`dbContext` ApplicationDbContext

Methods

CreateComment(Guid, Guid, CreateCommentRequest)

Creates a new comment on the specified post.

```
public Task<CommentDto> CreateComment(Guid userId, Guid postId,
CreateCommentRequest request)
```

Parameters

`userId` Guid

The ID of the user creating the comment.

`postId` Guid

The ID of the post being commented on.

`request` CreateCommentRequest

The comment content.

Returns

Task<CommentDto>

The newly created FractPal.Model.DTO.Comment.CommentDto.

Exceptions

KeyNotFoundException

Thrown when no post exists with `postId`.

DeleteComment(Guid, Guid)

Permanently deletes a comment. Only the comment's author may perform this operation.

```
public Task DeleteComment(Guid userId, Guid commentId)
```

Parameters

`userId` Guid

The ID of the requesting user.

`commentId` Guid

The ID of the comment to delete.

Returns

Task

Exceptions

KeyNotFoundException

Thrown when no comment exists with `commentId`.

UnauthorizedAccessException

Thrown when the requesting user is not the comment's author.

GetCommentById(Guid)

Retrieves a single comment by its unique identifier.

```
public Task<CommentDto> GetCommentById(Guid commentId)
```

Parameters

`commentId` Guid

The unique identifier of the comment.

Returns

Task<CommentDto>

The matching FractPal.Model.DTO.Comment.CommentDto.

Exceptions

KeyNotFoundException

Thrown when no comment exists with `commentId`.

GetPostComments(Guid)

Retrieves all comments on a given post, ordered chronologically.

```
public Task<List<CommentDto>> GetPostComments(Guid postId)
```

Parameters

`postId` Guid

The unique identifier of the post.

Returns

Task<List<CommentDto>>

A list of FractPal.Model.DTO.Comment.CommentDto for the post.

UpdateComment(Guid, Guid, UpdateCommentRequest)

Updates the content of an existing comment. Only the comment's author may edit it.

```
public Task<CommentDto> UpdateComment(Guid userId, Guid commentId,
UpdateCommentRequest request)
```

Parameters

`userId` Guid

The ID of the requesting user.

`commentId` Guid

The ID of the comment to update.

`request` UpdateCommentRequest

The updated comment content.

Returns

Task<CommentDto>

The updated FractPal.Model.DTO.Comment.CommentDto.

Exceptions

KeyNotFoundException

Thrown when no comment exists with `commentId`.

UnauthorizedAccessException

Thrown when the requesting user is not the comment's author.

Class FractalService

Namespace: FractPal.Service.Implementation

Assembly: FractPal.Service.dll

Implements fractal business logic for FractPal, including creation, retrieval, modification, forking, and deletion of fractals.

```
public class FractalService : IFractalService
```

Inheritance

object ← FractalService

Implements

IFractalService

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

FractalService(ApplicationDbContext)

Implements fractal business logic for FractPal, including creation, retrieval, modification, forking, and deletion of fractals.

```
public FractalService(ApplicationDbContext context)
```

Parameters

`context` ApplicationDbContext

Methods

CreateFractalAsync(Guid, CreateFractalRequest)

Creates a new fractal as a draft owned by the specified user.

```
public Task<FractalDto> CreateFractalAsync(Guid userId, CreateFractalRequest
request)
```

Parameters

`userId` Guid

The ID of the user creating the fractal.

`request` CreateFractalRequest

The fractal configuration data.

Returns

Task<FractalDto>

The newly created FractPal.Model.DTO.Fractal.FractalDto.

DeleteFractalAsync(Guid, Guid, bool)

Permanently deletes a fractal. Only the owner may perform this operation.

```
public Task DeleteFractalAsync(Guid fractalId, Guid userId, bool isAdmin = false)
```

Parameters

`fractalId` Guid

The unique identifier of the fractal to delete.

`userId` Guid

The ID of the requesting user.

`isAdmin` bool

Returns

Task

Exceptions

KeyNotFoundException

Thrown when no fractal exists with `fractalId`.

UnauthorizedAccessException

Thrown when the requesting user is not the owner.

ForkFractalAsync(Guid, Guid)

Creates a copy of an existing fractal under a new owner. The forked fractal starts as a draft regardless of the original's publish state.

```
public Task<FractalDto> ForkFractalAsync(Guid fractalId, Guid userId)
```

Parameters

`fractalId` Guid

The unique identifier of the fractal to fork.

`userId` Guid

The ID of the user who will own the fork.

Returns

Task<FractalDto>

The newly created forked FractPal.Model.DTO.Fractal.FractalDto.

Exceptions

KeyNotFoundException

Thrown when no fractal exists with `fractalId`.

GetFeedAsync(Guid, int, int)

Retrieves a paginated feed of all published fractals ordered by most recently published.

```
public Task<FractalFeedResponse> GetFeedAsync(Guid userId, int page = 1, int
pageSize = 20)
```

Parameters

`userId` Guid

The ID of the requesting user, used to compute `IsLikedByCurrentUser`.

`page` int

The 1-based page number.

`pageSize` int

The maximum number of fractals to return per page.

Returns

Task<FractalFeedResponse>

A FractPal.Model.DTO.Fractal.FractalFeedResponse containing fractals and pagination metadata.

GetFractalByIdAsync(Guid, Guid)

Retrieves a single fractal by its unique identifier.

```
public Task<FractalDto?> GetFractalByIdAsync(Guid fractalId, Guid currentUserId)
```

Parameters

`fractalId` Guid

The unique identifier of the fractal.

`currentUserId` Guid

The ID of the requesting user, used to compute `IsLikedByCurrentUser`.

Returns

Task<FractalDto>

The matching FractPal.Model.DTO.Fractal.FractalDto, or `null` if not found.

GetPublishedFractalsByUserAsync(Guid, Guid)

Retrieves only the published fractals belonging to a specific user.

```
public Task<List<FractalDto>> GetPublishedFractalsByUserAsync(Guid userId,
Guid currentUserId)
```

Parameters

`userId` Guid

The ID of the user whose published fractals to retrieve.

`currentUserId` Guid

The ID of the requesting user, used to compute `IsLikedByCurrentUser`.

Returns

Task<List<FractalDto>>

A list of published FractPal.Model.DTO.Fractal.FractalDto for the specified user.

GetUserFractalsAsync(Guid, Guid)

Retrieves all fractals (published and draft) belonging to a specific user.

```
public Task<List<FractalDto>> GetUserFractalsAsync(Guid userId, Guid currentUserId)
```

Parameters

`userId` Guid

The ID of the user whose fractals to retrieve.

`currentUserId` Guid

The ID of the requesting user, used to compute `IsLikedByCurrentUser`.

Returns

Task<List<FractalDto>>

A list of FractPal.Model.DTO.Fractal.FractalDto for the specified user.

UpdateFractalAsync(Guid, Guid, UpdateFractalRequest)

Updates an existing fractal. If the requesting user is not the owner, a forked copy is created instead and the original remains unchanged.

```
public Task<FractalDto> UpdateFractalAsync(Guid fractalId, Guid userId,
UpdateFractalRequest request)
```

Parameters

`fractalId` Guid

The unique identifier of the fractal to update.

`userId` Guid

The ID of the requesting user.

`request` UpdateFractalRequest

The updated fractal configuration data.

Returns

Task<FractalDto>

The updated FractPal.Model.DTO.Fractal.FractalDto if the user is the owner, or a new forked FractPal.Model.DTO.Fractal.FractalDto otherwise.

Exceptions

KeyNotFoundException

Thrown when no fractal exists with `fractalId`.

Class JwtService

Namespace: FractPal.Service.Implementation

Assembly: FractPal.Service.dll

Generates signed JWT access tokens for authenticated FractPal users, including role claims. Token settings (secret, issuer, audience, expiry) are resolved from environment variables first, falling back to `appsettings.json`.

```
public class JwtService : IJwtService
```

Inheritance

object ← JwtService

Implements

IJwtService

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

JwtService(IConfiguration, UserManager<FractPalUser>)

Generates signed JWT access tokens for authenticated FractPal users, including role claims. Token settings (secret, issuer, audience, expiry) are resolved from environment variables first, falling back to `appsettings.json`.

```
public JwtService(IConfiguration configuration, UserManager<FractPalUser>
userManager)
```

Parameters

`configuration` IConfiguration

`userManager` UserManager<FractPalUser>

Methods

GenerateToken(FractPalUser)

Generates a signed JWT access token for the given user, including their role claims.

```
public Task<string> GenerateToken(FractPalUser user)
```

Parameters

`user` FractPalUser

The authenticated FractPal.Model.Entities.FractPalUser for whom the token is issued.

Returns

Task<string>

A signed JWT string.

Class LikeService

Namespace: FractPal.Service.Implementation

Assembly: FractPal.Service.dll

Implements like/unlike toggle logic for fractals and posts. Likes are stored at the fractal level; post likes resolve to the underlying fractal.

```
public class LikeService : ILikeService
```

Inheritance

object ← LikeService

Implements

ILikeService

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

LikeService(ApplicationDbContext)

Implements like/unlike toggle logic for fractals and posts. Likes are stored at the fractal level; post likes resolve to the underlying fractal.

```
public LikeService(ApplicationDbContext dbContext)
```

Parameters

`dbContext` ApplicationDbContext

Methods

ToggleLikeAsync(Guid, Guid)

Toggles the like state for a fractal by the specified user.

```
public Task<bool> ToggleLikeAsync(Guid fractalId, Guid userId)
```

Parameters

`fractalId` Guid

The unique identifier of the fractal.

`userId` Guid

The ID of the user performing the action.

Returns

Task<bool>

`true` if the fractal is now liked; `false` if the like was removed.

Exceptions

KeyNotFoundException

Thrown when no fractal exists with `fractalId`.

ToggleLikePostAsync(Guid, Guid)

Toggles the like state for a post by resolving it to its underlying fractal.

```
public Task<bool> ToggleLikePostAsync(Guid postId, Guid userId)
```

Parameters

`postId` Guid

The unique identifier of the post.

`userId` Guid

The ID of the user performing the action.

Returns

Task<bool>

`true` if the post's fractal is now liked; `false` if the like was removed.

Exceptions

KeyNotFoundException

Thrown when no post exists with `postId`.

Class PostService

Namespace: FractPal.Service.Implementation

Assembly: FractPal.Service.dll

Implements post business logic for FractPal, including publishing fractals as posts, feed retrieval, and post lifecycle management.

```
public class PostService : IPostService
```

Inheritance

object ← PostService

Implements

IPostService

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

PostService(ApplicationDbContext)

Implements post business logic for FractPal, including publishing fractals as posts, feed retrieval, and post lifecycle management.

```
public PostService(ApplicationDbContext dbContext)
```

Parameters

`dbContext` ApplicationDbContext

Methods

DeletePostAsync(Guid, Guid, bool)

Permanently deletes a post. Admins may delete any post; regular users may only delete their own.

```
public Task DeletePostAsync(Guid postId, Guid userId, bool isAdmin = false)
```

Parameters

`postId` Guid

The unique identifier of the post to delete.

`userId` Guid

The ID of the requesting user.

`isAdmin` bool

Whether the requesting user has admin privileges.

Returns

Task

Exceptions

KeyNotFoundException

Thrown when no post exists with `postId`.

UnauthorizedAccessException

Thrown when a non-admin user attempts to delete another user's post.

GetFeedAsync(Guid, int, int)

Retrieves a paginated feed of all posts, ordered by most recently created.

```
public Task<PostFeedResponse> GetFeedAsync(Guid userId, int page = 1, int pageSize
= 20)
```

Parameters

`userId` Guid

The ID of the requesting user, used to compute `IsLikedByCurrentUser`.

`page` int

The 1-based page number.

`pageSize` int

The maximum number of posts to return per page.

Returns

Task<PostFeedResponse>

A FractPal.Model.DTO.Post.PostFeedResponse containing posts and pagination metadata.

GetPostByIdAsync(Guid, Guid)

Retrieves a single post by its unique identifier.

```
public Task<PostDto?> GetPostByIdAsync(Guid postId, Guid currentUserId)
```

Parameters

`postId` Guid

The unique identifier of the post.

`currentUserId` Guid

The ID of the requesting user, used to compute `IsLikedByCurrentUser`.

Returns

Task<PostDto>

The matching FractPal.Model.DTO.Post.PostDto, or `null` if not found.

GetUserPostsAsync(Guid, Guid)

Retrieves all posts authored by a specific user.

```
public Task<List<PostDto>> GetUserPostsAsync(Guid userId, Guid currentUserId)
```

Parameters

`userId` Guid

The ID of the user whose posts to retrieve.

`currentUserId` Guid

The ID of the requesting user, used to compute `IsLikedByCurrentUser`.

Returns

Task<List<PostDto>>

A list of FractPal.Model.DTO.Post.PostDto for the specified user.

PublishFractalAsync(Guid, Guid, CreatePostRequest)

Publishes a fractal as a post. Only the fractal's owner may publish it.

```
public Task<PostDto> PublishFractalAsync(Guid fractalId, Guid userId,
CreatePostRequest request)
```

Parameters

`fractalId` Guid

The unique identifier of the fractal to publish.

`userId` Guid

The ID of the requesting user.

`request` CreatePostRequest

The post metadata including name and description.

Returns

Task<PostDto>

The newly created FractPal.Model.DTO.Post.PostDto.

Exceptions

KeyNotFoundException

Thrown when no fractal exists with `fractalId`.

UnauthorizedAccessException

Thrown when the requesting user is not the fractal's owner.

UnpublishFractalAsync(Guid, Guid)

Removes the published post associated with a fractal. Only the post's author may unpublish it.

```
public Task UnpublishFractalAsync(Guid fractalId, Guid userId)
```

Parameters

`fractalId` Guid

The unique identifier of the fractal whose post to remove.

`userId` Guid

The ID of the requesting user.

Returns

Task

Exceptions

KeyNotFoundException

Thrown when no matching post exists.

UnauthorizedAccessException

Thrown when the requesting user is not the post's author.

UpdatePostAsync(Guid, Guid, UpdatePostRequest)

Updates the metadata (name and description) of an existing post. Only the post's author may update it.

```
public Task<PostDto> UpdatePostAsync(Guid postId, Guid userId,
UpdatePostRequest request)
```

Parameters

`postId` Guid

The unique identifier of the post to update.

`userId` Guid

The ID of the requesting user.

`request` UpdatePostRequest

The updated post metadata.

Returns

Task<PostDto>

The updated FractPal.Model.DTO.Post.PostDto.

Exceptions

KeyNotFoundException

Thrown when no post exists with `postId`.

UnauthorizedAccessException

Thrown when the requesting user is not the post's author.

Class RefreshTokenService

Namespace: FractPal.Service.Implementation

Assembly: FractPal.Service.dll

Manages refresh token lifecycle: generation, validation, lookup, and revocation. Tokens are cryptographically random, stored in the repository, and expire after 7 days.

```
public class RefreshTokenService : IRefreshTokenService
```

Inheritance

object ← RefreshTokenService

Implements

IRefreshTokenService

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

RefreshTokenService(IRepository<RefreshToken>)

Manages refresh token lifecycle: generation, validation, lookup, and revocation. Tokens are cryptographically random, stored in the repository, and expire after 7 days.

```
public RefreshTokenService(IRepository<RefreshToken> refreshTokenRepository)
```

Parameters

`refreshTokenRepository` IRepository<RefreshToken>

Methods

GenerateRefreshToken(IdentityUser)

Generates a new cryptographically secure refresh token for the specified user and persists it for future validation.

```
public Task<string> GenerateRefreshToken(IdentityUser user)
```

Parameters

`user` IdentityUser

The user to generate a refresh token for.

Returns

Task<string>

The generated refresh token string.

GetUserIdByRefreshToken(string)

Looks up the user ID associated with the given refresh token.

```
public Task<Guid?> GetUserIdByRefreshToken(string refreshToken)
```

Parameters

`refreshToken` string

The refresh token to look up.

Returns

Task<Guid?>

The associated user's Guid, or `null` if the token is not found or expired.

InvalidateRefreshToken(string)

Revokes a refresh token so it can no longer be used to obtain new access tokens.

```
public Task InvalidateRefreshToken(string refreshToken)
```

Parameters

`refreshToken` string

The refresh token to invalidate.

Returns

Task

ValidateRefreshToken(Guid, string)

Validates whether the given refresh token is active and belongs to the specified user.

```
public Task<bool> ValidateRefreshToken(Guid userId, string refreshToken)
```

Parameters

`userId` Guid

The ID of the user claiming the token.

`refreshToken` string

The refresh token to validate.

Returns

Task<bool>

`true` if the token is valid; otherwise `false`.

Class UserService

Namespace: FractPal.Service.Implementation

Assembly: FractPal.Service.dll

Implements user profile and social features for FractPal, including profile retrieval and updates, follow/unfollow toggling, and username search.

```
public class UserService : IUserService
```

Inheritance

object ← UserService

Implements

IUserService

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

UserService(ApplicationDbContext)

Implements user profile and social features for FractPal, including profile retrieval and updates, follow/unfollow toggling, and username search.

```
public UserService(ApplicationDbContext context)
```

Parameters

`context` ApplicationDbContext

Methods

GetProfileAsync(string, string)

Retrieves the profile of a user by their ID.

```
public Task<UserProfileDto> GetProfileAsync(string userId, string currentUserId)
```

Parameters

`userId` string

The ID of the user whose profile to retrieve.

`currentUserId` string

The ID of the requesting user, used to compute `IsFollowedByCurrentUser`.

Returns

Task<UserProfileDto>

A FractPal.Model.DTO.User.UserProfileDto for the requested user.

Exceptions

KeyNotFoundException

Thrown when no user exists with `userId`.

SearchUsersAsync(string, string)

Searches for users whose usernames contain the given query string (case-insensitive). Returns up to 20 results.

```
public Task<List<UserSearchDto>> SearchUsersAsync(string query,
string currentUserId)
```

Parameters

`query` string

The search term to match against usernames.

`currentUserId` string

The ID of the requesting user, used to compute `IsFollowedByCurrentUser`.

Returns

Task<List<UserSearchDto>>

A list of up to 20 matching FractPal.Model.DTO.User.UserSearchDto.

ToggleFollowAsync(string, string)

Toggles the follow relationship from `followerId` to `followingId`. If already following, unfollows. If not following, follows.

```
public Task<bool> ToggleFollowAsync(string followerId, string followingId)
```

Parameters

`followerId` string

The ID of the user initiating the follow/unfollow.

`followingId` string

The ID of the user being followed or unfollowed.

Returns

Task<bool>

`true` if the result is now following; `false` if unfollowed.

Exceptions

InvalidOperationException

Thrown when a user attempts to follow themselves.

UpdateProfileAsync(string, UpdateProfileRequest)

Updates mutable profile fields (e.g. bio) for the specified user.

```
public Task<UserProfileDto> UpdateProfileAsync(string userId,
UpdateProfileRequest request)
```

Parameters

`userId` string

The ID of the user to update.

`request` UpdateProfileRequest

The updated profile data.

Returns

Task<UserProfileDto>

The updated FractPal.Model.DTO.User.UserProfileDto.

Exceptions

KeyNotFoundException

Thrown when no user exists with `userId`.

Namespace FractPal.Service.Interface

Interfaces

IAuthService
: Provides authentication operations for user login and registration.

ICommentService
: Provides business logic for creating, retrieving, updating, and deleting comments on FractPal posts.

IFractalService
: Provides business logic for fractal creation, retrieval, modification, and social features.

IJwtService
: Provides JWT token generation for authenticated users.

ILikeService
: Provides like/unlike toggle operations for fractals and posts.

IPostService
: Provides business logic for publishing fractals as posts, retrieving feeds, and managing post lifecycle on FractPal.

IRefreshTokenService
: Manages refresh token lifecycle: generation, validation, and invalidation.

IRepository<T>
: Defines a generic repository interface for common data access operations

IUserService
: Provides user profile management and social features such as following and search.

Interface IAuthService

Namespace: FractPal.Service.Interface

Assembly: FractPal.Service.dll

Provides authentication operations for user login and registration.

```
public interface IAuthService
```

Methods

Login(HttpContext, LoginRequest)

Authenticates a user with the provided credentials and issues an access token. Sets any necessary cookies or session state via `context`.

```
Task<LoginResponse> Login(HttpContext context, LoginRequest request)
```

Parameters

`context` HttpContext

The current HTTP context, used to set authentication cookies if applicable.

`request` LoginRequest

The login credentials containing email and password.

Returns

Task<LoginResponse>

A FractPal.Model.DTO.Auth.LoginResponse containing the access token and user information.

Exceptions

UnauthorizedAccessException

Thrown when the email/password combination is invalid.

Register(RegistrationRequest)

Registers a new user account with the provided details.

```
Task<RegistrationResponse> Register(RegistrationRequest request)
```

Parameters

`request` RegistrationRequest

The registration data including username, email and password.

Returns

Task<RegistrationResponse>

A FractPal.Model.DTO.Auth.RegistrationResponse confirming the created account.

Exceptions

InvalidOperationException

Thrown when the email or username is already in use.

Interface ICommentService

Namespace: FractPal.Service.Interface

Assembly: FractPal.Service.dll

Provides business logic for creating, retrieving, updating, and deleting comments on FractPal posts.

```
public interface ICommentService
```

Methods

CreateComment(Guid, Guid, CreateCommentRequest)

Creates a new comment on the specified post.

```
Task<CommentDto> CreateComment(Guid userId, Guid postId, CreateCommentRequest
request)
```

Parameters

`userId` Guid

The ID of the user creating the comment.

`postId` Guid

The ID of the post being commented on.

`request` CreateCommentRequest

The comment content.

Returns

Task<CommentDto>

The newly created FractPal.Model.DTO.Comment.CommentDto.

Exceptions

KeyNotFoundException

Thrown when no post exists with `postId`.

DeleteComment(Guid, Guid)

Permanently deletes a comment. Only the comment's author may perform this operation.

```
Task DeleteComment(Guid userId, Guid commentId)
```

Parameters

`userId` Guid

The ID of the requesting user.

`commentId` Guid

The ID of the comment to delete.

Returns

Task

Exceptions

KeyNotFoundException

Thrown when no comment exists with `commentId`.

UnauthorizedAccessException

Thrown when the requesting user is not the comment's author.

GetCommentById(Guid)

Retrieves a single comment by its unique identifier.

```
Task<CommentDto> GetCommentById(Guid commentId)
```

Parameters

`commentId` Guid

The unique identifier of the comment.

Returns

Task<CommentDto>

The matching FractPal.Model.DTO.Comment.CommentDto.

Exceptions

KeyNotFoundException

Thrown when no comment exists with `commentId`.

GetPostComments(Guid)

Retrieves all comments on a given post, ordered chronologically.

```
Task<List<CommentDto>> GetPostComments(Guid postId)
```

Parameters

`postId` Guid

The unique identifier of the post.

Returns

Task<List<CommentDto>>

A list of FractPal.Model.DTO.Comment.CommentDto for the post.

UpdateComment(Guid, Guid, UpdateCommentRequest)

Updates the content of an existing comment. Only the comment's author may edit it.

```
Task<CommentDto> UpdateComment(Guid userId, Guid commentId,
UpdateCommentRequest request)
```

Parameters

`userId` Guid

The ID of the requesting user.

`commentId` Guid

The ID of the comment to update.

`request` UpdateCommentRequest

The updated comment content.

Returns

Task<CommentDto>

The updated FractPal.Model.DTO.Comment.CommentDto.

Exceptions

KeyNotFoundException

Thrown when no comment exists with `commentId`.

UnauthorizedAccessException

Thrown when the requesting user is not the comment's author.

Interface IFractalService

Namespace: FractPal.Service.Interface

Assembly: FractPal.Service.dll

Provides business logic for fractal creation, retrieval, modification, and social features.

```
public interface IFractalService
```

Methods

CreateFractalAsync(Guid, CreateFractalRequest)

Creates a new fractal as a draft owned by the specified user.

```
Task<FractalDto> CreateFractalAsync(Guid userId, CreateFractalRequest request)
```

Parameters

`userId` Guid

The ID of the user creating the fractal.

`request` CreateFractalRequest

The fractal configuration data.

Returns

Task<FractalDto>

The newly created FractPal.Model.DTO.Fractal.FractalDto.

DeleteFractalAsync(Guid, Guid, bool)

Permanently deletes a fractal. Only the owner may perform this operation.

```
Task DeleteFractalAsync(Guid fractalId, Guid userId, bool isAdmin = false)
```

Parameters

`fractalId` Guid

The unique identifier of the fractal to delete.

`userId` Guid

The ID of the requesting user.

`isAdmin` bool

Returns

Task

Exceptions

KeyNotFoundException

Thrown when no fractal exists with `fractalId`.

UnauthorizedAccessException

Thrown when the requesting user is not the owner.

ForkFractalAsync(Guid, Guid)

Creates a copy of an existing fractal under a new owner. The forked fractal starts as a draft regardless of the original's publish state.

```
Task<FractalDto> ForkFractalAsync(Guid fractalId, Guid userId)
```

Parameters

`fractalId` Guid

The unique identifier of the fractal to fork.

`userId` Guid

The ID of the user who will own the fork.

Returns

Task<FractalDto>

The newly created forked FractPal.Model.DTO.Fractal.FractalDto.

Exceptions

KeyNotFoundException

Thrown when no fractal exists with `fractalId`.

GetFeedAsync(Guid, int, int)

Retrieves a paginated feed of all published fractals ordered by most recently published.

```
Task<FractalFeedResponse> GetFeedAsync(Guid userId, int page = 1, int pageSize = 20)
```

Parameters

`userId` Guid

The ID of the requesting user, used to compute `IsLikedByCurrentUser`.

`page` int

The 1-based page number.

`pageSize` int

The maximum number of fractals to return per page.

Returns

Task<FractalFeedResponse>

A FractPal.Model.DTO.Fractal.FractalFeedResponse containing fractals and pagination metadata.

GetFractalByIdAsync(Guid, Guid)

Retrieves a single fractal by its unique identifier.

```
Task<FractalDto?> GetFractalByIdAsync(Guid fractalId, Guid currentUserId)
```

Parameters

`fractalId` Guid

The unique identifier of the fractal.

`currentUserId` Guid

The ID of the requesting user, used to compute `IsLikedByCurrentUser`.

Returns

Task<FractalDto>

The matching FractPal.Model.DTO.Fractal.FractalDto, or `null` if not found.

GetPublishedFractalsByUserAsync(Guid, Guid)

Retrieves only the published fractals belonging to a specific user.

```
Task<List<FractalDto>> GetPublishedFractalsByUserAsync(Guid userId,
Guid currentUserId)
```

Parameters

`userId` Guid

The ID of the user whose published fractals to retrieve.

`currentUserId` Guid

The ID of the requesting user, used to compute `IsLikedByCurrentUser`.

Returns

Task<List<FractalDto>>

A list of published FractPal.Model.DTO.Fractal.FractalDto for the specified user.

GetUserFractalsAsync(Guid, Guid)

Retrieves all fractals (published and draft) belonging to a specific user.

```
Task<List<FractalDto>> GetUserFractalsAsync(Guid userId, Guid currentUserId)
```

Parameters

`userId` Guid

The ID of the user whose fractals to retrieve.

`currentUserId` Guid

The ID of the requesting user, used to compute `IsLikedByCurrentUser`.

Returns

Task<List<FractalDto>>

A list of FractPal.Model.DTO.Fractal.FractalDto for the specified user.

UpdateFractalAsync(Guid, Guid, UpdateFractalRequest)

Updates an existing fractal. If the requesting user is not the owner, a forked copy is created instead and the original remains unchanged.

```
Task<FractalDto> UpdateFractalAsync(Guid fractalId, Guid userId,
UpdateFractalRequest request)
```

Parameters

`fractalId` Guid

The unique identifier of the fractal to update.

`userId` Guid

The ID of the requesting user.

`request` UpdateFractalRequest

The updated fractal configuration data.

Returns

Task<FractalDto>

The updated FractPal.Model.DTO.Fractal.FractalDto if the user is the owner, or a new forked FractPal.Model.DTO.Fractal.FractalDto otherwise.

Exceptions

KeyNotFoundException

Thrown when no fractal exists with `fractalId`.

Interface IJwtService

Namespace: FractPal.Service.Interface

Assembly: FractPal.Service.dll

Provides JWT token generation for authenticated users.

```
public interface IJwtService
```

Methods

GenerateToken(FractPalUser)

Generates a signed JWT access token for the given user, including their role claims.

```
Task<string> GenerateToken(FractPalUser user)
```

Parameters

`user` FractPalUser

The authenticated FractPal.Model.Entities.FractPalUser for whom the token is issued.

Returns

Task<string>

A signed JWT string.

Interface ILikeService

Namespace: FractPal.Service.Interface

Assembly: FractPal.Service.dll

Provides like/unlike toggle operations for fractals and posts.

```
public interface ILikeService
```

Methods

ToggleLikeAsync(Guid, Guid)

Toggles the like state for a fractal by the specified user.

```
Task<bool> ToggleLikeAsync(Guid fractalId, Guid userId)
```

Parameters

`fractalId` Guid

The unique identifier of the fractal.

`userId` Guid

The ID of the user performing the action.

Returns

Task<bool>

`true` if the fractal is now liked; `false` if the like was removed.

Exceptions

KeyNotFoundException

Thrown when no fractal exists with `fractalId`.

ToggleLikePostAsync(Guid, Guid)

Toggles the like state for a post by resolving it to its underlying fractal.

```
Task<bool> ToggleLikePostAsync(Guid postId, Guid userId)
```

Parameters

`postId` Guid

The unique identifier of the post.

`userId` Guid

The ID of the user performing the action.

Returns

Task<bool>

`true` if the post's fractal is now liked; `false` if the like was removed.

Exceptions

KeyNotFoundException

Thrown when no post exists with `postId`.

Interface IPostService

Namespace: FractPal.Service.Interface

Assembly: FractPal.Service.dll

Provides business logic for publishing fractals as posts, retrieving feeds, and managing post lifecycle on FractPal.

```
public interface IPostService
```

Methods

DeletePostAsync(Guid, Guid, bool)

Permanently deletes a post. Admins may delete any post; regular users may only delete their own.

```
Task DeletePostAsync(Guid postId, Guid userId, bool isAdmin = false)
```

Parameters

`postId` Guid

The unique identifier of the post to delete.

`userId` Guid

The ID of the requesting user.

`isAdmin` bool

Whether the requesting user has admin privileges.

Returns

Task

Exceptions

KeyNotFoundException

Thrown when no post exists with `postId`.

UnauthorizedAccessException

Thrown when a non-admin user attempts to delete another user's post.

GetFeedAsync(Guid, int, int)

Retrieves a paginated feed of all posts, ordered by most recently created.

```
Task<PostFeedResponse> GetFeedAsync(Guid userId, int page = 1, int pageSize = 20)
```

Parameters

`userId` Guid

The ID of the requesting user, used to compute `IsLikedByCurrentUser`.

`page` int

The 1-based page number.

`pageSize` int

The maximum number of posts to return per page.

Returns

Task<PostFeedResponse>

A FractPal.Model.DTO.Post.PostFeedResponse containing posts and pagination metadata.

GetPostByIdAsync(Guid, Guid)

Retrieves a single post by its unique identifier.

```
Task<PostDto?> GetPostByIdAsync(Guid postId, Guid currentUserId)
```

Parameters

`postId` Guid

The unique identifier of the post.

`currentUserId` Guid

The ID of the requesting user, used to compute `IsLikedByCurrentUser`.

Returns

Task<PostDto>

The matching FractPal.Model.DTO.Post.PostDto, or `null` if not found.

GetUserPostsAsync(Guid, Guid)

Retrieves all posts authored by a specific user.

```
Task<List<PostDto>> GetUserPostsAsync(Guid userId, Guid currentUserId)
```

Parameters

`userId` Guid

The ID of the user whose posts to retrieve.

`currentUserId` Guid

The ID of the requesting user, used to compute `IsLikedByCurrentUser`.

Returns

Task<List<PostDto>>

A list of FractPal.Model.DTO.Post.PostDto for the specified user.

PublishFractalAsync(Guid, Guid, CreatePostRequest)

Publishes a fractal as a post. Only the fractal's owner may publish it.

```
Task<PostDto> PublishFractalAsync(Guid fractalId, Guid userId,
CreatePostRequest request)
```

Parameters

`fractalId` Guid

The unique identifier of the fractal to publish.

`userId` Guid

The ID of the requesting user.

`request` CreatePostRequest

The post metadata including name and description.

Returns

Task<PostDto>

The newly created FractPal.Model.DTO.Post.PostDto.

Exceptions

KeyNotFoundException

Thrown when no fractal exists with `fractalId`.

UnauthorizedAccessException

Thrown when the requesting user is not the fractal's owner.

UnpublishFractalAsync(Guid, Guid)

Removes the published post associated with a fractal. Only the post's author may unpublish it.

```
Task UnpublishFractalAsync(Guid fractalId, Guid userId)
```

Parameters

`fractalId` Guid

The unique identifier of the fractal whose post to remove.

`userId` Guid

The ID of the requesting user.

Returns

Task

Exceptions

KeyNotFoundException

Thrown when no matching post exists.

UnauthorizedAccessException

Thrown when the requesting user is not the post's author.

UpdatePostAsync(Guid, Guid, UpdatePostRequest)

Updates the metadata (name and description) of an existing post. Only the post's author may update it.

```
Task<PostDto> UpdatePostAsync(Guid postId, Guid userId, UpdatePostRequest request)
```

Parameters

`postId` Guid

The unique identifier of the post to update.

`userId` Guid

The ID of the requesting user.

`request` UpdatePostRequest

The updated post metadata.

Returns

Task<PostDto>

The updated FractPal.Model.DTO.Post.PostDto.

Exceptions

KeyNotFoundException

Thrown when no post exists with `postId`.

UnauthorizedAccessException

Thrown when the requesting user is not the post's author.

Interface IRefreshTokenService

Namespace: FractPal.Service.Interface

Assembly: FractPal.Service.dll

Manages refresh token lifecycle: generation, validation, and invalidation.

```
public interface IRefreshTokenService
```

Methods

GenerateRefreshToken(IdentityUser)

Generates a new cryptographically secure refresh token for the specified user and persists it for future validation.

```
Task<string> GenerateRefreshToken(IdentityUser user)
```

Parameters

`user` IdentityUser

The user to generate a refresh token for.

Returns

Task<string>

The generated refresh token string.

GetUserIdByRefreshToken(string)

Looks up the user ID associated with the given refresh token.

```
Task<Guid?> GetUserIdByRefreshToken(string refreshToken)
```

Parameters

`refreshToken` string

The refresh token to look up.

Returns

Task<Guid?>

The associated user's Guid, or `null` if the token is not found or expired.

InvalidateRefreshToken(string)

Revokes a refresh token so it can no longer be used to obtain new access tokens.

```
Task InvalidateRefreshToken(string refreshToken)
```

Parameters

`refreshToken` string

The refresh token to invalidate.

Returns

Task

ValidateRefreshToken(Guid, string)

Validates whether the given refresh token is active and belongs to the specified user.

```
Task<bool> ValidateRefreshToken(Guid userId, string refreshToken)
```

Parameters

`userId` Guid

The ID of the user claiming the token.

`refreshToken` string

The refresh token to validate.

Returns

Task<bool>

`true` if the token is valid; otherwise `false`.

Interface IRepository<T>

Namespace: FractPal.Service.Interface

Assembly: FractPal.Service.dll

Defines a generic repository interface for common data access operations

```
public interface IRepository<T> where T : class
```

Type Parameters

`T`

The type of the entity

Methods

AddAsync(T)

Add a new entity to the repository

```
Task AddAsync(T entity)
```

Parameters

`entity` T

The entity to add

Returns

Task

CommitAsync()

Commits all changes made in the current unit of work

```
Task<int> CommitAsync()
```

Returns

Task<int>

The number of affected rows

FilterAsync(Expression<Func<T, bool>>, params Expression<Func<T, object>>[])

Finds all entities matching the specified predicate

```
Task<IEnumerable<T>> FilterAsync(Expression<Func<T, bool>> predicate, params
Expression<Func<T, object>>[] includes)
```

Parameters

`predicate` Expression<Func<T, bool>>

The filtering condition

`includes` Expression<Func<T, object>>[]

Optional navigation properties to include

Returns

Task<IEnumerable<T>>

A list of matching entities

GetAllAsync(params Expression<Func<T, object>>[])

Retrieves all entities

```
Task<IEnumerable<T>> GetAllAsync(params Expression<Func<T, object>>[] includes)
```

Parameters

`includes` Expression<Func<T, object>>[]

Optional navigation properties to include

Returns

Task<IEnumerable<T>>

A list of all entities

GetByIdAsync(Guid, params Expression<Func<T, object>>[])

Retrives single entity by its GUID

```
Task<T?> GetByIdAsync(Guid id, params Expression<Func<T, object>>[] includes)
```

Parameters

`id` Guid

The unique identifier of the entity

`includes` Expression<Func<T, object>>[]

Optional navigation properties to include

Returns

Task<T>

The entity if found; otherwise, null

Query()

Provides a queryable interface for the repository, allowing for LINQ queries

```
IQueryable<T> Query()
```

Returns

IQueryable<T>

An IQueryable<T> for entity type.

Remove(T)

Removes an entity from the repository

```
void Remove(T entity)
```

Parameters

`entity` T

The entity to remove

Update(T)

Updates an existing entity

```
void Update(T entity)
```

Parameters

`entity` T

The entity to update

Interface IUserService

Namespace: FractPal.Service.Interface

Assembly: FractPal.Service.dll

Provides user profile management and social features such as following and search.

```
public interface IUserService
```

Methods

GetProfileAsync(string, string)

Retrieves the profile of a user by their ID.

```
Task<UserProfileDto> GetProfileAsync(string userId, string currentUserId)
```

Parameters

`userId` string

The ID of the user whose profile to retrieve.

`currentUserId` string

The ID of the requesting user, used to compute `IsFollowedByCurrentUser`.

Returns

Task<UserProfileDto>

A FractPal.Model.DTO.User.UserProfileDto for the requested user.

Exceptions

KeyNotFoundException

Thrown when no user exists with `userId`.

SearchUsersAsync(string, string)

Searches for users whose usernames contain the given query string (case-insensitive). Returns up to 20 results.

```
Task<List<UserSearchDto>> SearchUsersAsync(string query, string currentUserId)
```

Parameters

`query` string

The search term to match against usernames.

`currentUserId` string

The ID of the requesting user, used to compute `IsFollowedByCurrentUser`.

Returns

Task<List<UserSearchDto>>

A list of up to 20 matching FractPal.Model.DTO.User.UserSearchDto.

ToggleFollowAsync(string, string)

Toggles the follow relationship from `followerId` to `followingId`. If already following, unfollows. If not following, follows.

```
Task<bool> ToggleFollowAsync(string followerId, string followingId)
```

Parameters

`followerId` string

The ID of the user initiating the follow/unfollow.

`followingId` string

The ID of the user being followed or unfollowed.

Returns

Task<bool>

`true` if the result is now following; `false` if unfollowed.

Exceptions

InvalidOperationException

Thrown when a user attempts to follow themselves.

UpdateProfileAsync(string, UpdateProfileRequest)

Updates mutable profile fields (e.g. bio) for the specified user.

```
Task<UserProfileDto> UpdateProfileAsync(string userId, UpdateProfileRequest request)
```

Parameters

`userId` string

The ID of the user to update.

`request` UpdateProfileRequest

The updated profile data.

Returns

Task<UserProfileDto>

The updated FractPal.Model.DTO.User.UserProfileDto.

Exceptions

KeyNotFoundException

Thrown when no user exists with `userId`.